

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE
KHOUREBGA

Département Mathématiques et Informatique

Projet libre

Filière : Génie informatique

Projet de Gestion de Laboratoire

EL AKHIRI Oussama
LAMCHANNEQ ZAKARIA

Présenté et soutenu publiquement le (06/01/2025)

Composition du jury :

Encadrant : Mr. HAFIDI Imad

Encadrant : Mme. SOUSSI Nassima

Encadrant : Mme. OURDOU Amal

Encadrant : Mme. KAROUM Bouchra

Résumé

Dans ce projet, nous avons conçu et développé un système complet de gestion de laboratoire en nous appuyant sur une architecture microservices avec Spring Boot pour le back-end et Angular pour le front-end. Nous avons mis en place un Config Server, un Discovery Server et une API Gateway sécurisée via JWT pour la gestion des rôles et droits d'accès. Par ailleurs, Kafka a été utilisé pour la communication asynchrone, et nous avons intégré SonarQube pour l'analyse de la qualité du code, JMeter pour les tests de performance, et Selenium pour les tests automatisés de l'interface utilisateur. Nous avons effectué des tests unitaires et d'intégration approfondis afin d'assurer la qualité du code et instauré une intégration continue et un déploiement continu (CI/CD) grâce à Docker et GitHub Actions. Ces outils nous ont permis de maintenir des standards élevés de développement, d'évaluer les performances du système sous charge, et de garantir la fiabilité de l'interface utilisateur. Ce projet a considérablement élargi nos compétences techniques tout en renforçant notre capacité à collaborer efficacement.

Mots-clés : microservices, Spring Boot, Angular, JWT, Docker, CI/CD, Kafka, SonarQube, JMeter, Selenium, tests unitaires, tests d'intégration

Abstract

In this project, we designed and implemented a comprehensive Laboratory Management System based on a microservices architecture using Spring Boot for the backend and Angular for the frontend. Our setup included a Config Server, Discovery Server, and a Gateway secured by JWT for role-based access control. We leveraged Kafka for asynchronous messaging and employed SonarQube for code quality analysis, JMeter for performance testing, and Selenium for automated UI testing. We performed extensive unit and integration testing to ensure code quality and implemented a CI/CD pipeline with Docker and GitHub Actions. Additionally, we incorporated SonarQube to maintain high code standards, JMeter to assess system performance under load, and Selenium to validate the user interface through automated tests. This project significantly enhanced our technical expertise and strengthened our teamwork skills by adopting modern development practices and tools.

Keywords : microservices, Spring Boot, Angular, JWT, Docker, CI/CD, Kafka, SonarQube, JMeter, Selenium, tests unitaires, tests d'intégration

الملخص

في هذا المشروع، قمنا بتصميم وتطوير نظام متكامل لإدارة المختبرات معتمد على بنية الميكروسيرفس باستخدام Spring Boot في الجانب الخلفي و Angular في الواجهة الأمامية. تضمن المشروع إعداد خادم التكوين (Config Server)، خدمة الاكتشاف (Discovery Server)، بوابة آمنة (Gateway) تعتمد على JWT لإدارة الصلاحيات، وأيضاً استخدام Kafka لإرسال الرسائل بشكل غير متزامن. بالإضافة إلى ذلك، استخدمنا SonarQube لتحليل جودة الكود، JMeter لاختبارات الأداء، Selenium لاختبارات واجهة المستخدم التلقائية. قمنا بإنجاز اختبارات وحدات وتكامل مكثفة لضمان جودة الكود وإعداد بيئة CI/CD عبر Docker و GitHub Actions. أسهم هذا المشروع في صقل مهاراتنا التقنية وتحسين قدرتنا على العمل الجماعي عبر تبني ممارسات حديثة وعدة أدوات متقدمة.

- الكلمات المفتاحية: ميكروسيرفس، Spring Boot، Angular، JWT، Docker، CI/CD، Kafka، SonarQube، Selenium، JMeter، اختبارات وحدات، اختبارات تكامل

Remerciements

Nous remercions Dieu Tout-Puissant pour nous avoir accordé la santé et la détermination nécessaires à l'accomplissement de ce projet.

Nous souhaitons exprimer toute notre gratitude à nos professeurs pour leur précieux accompagnement tout au long de ce projet. Grâce à leurs éclaircissements, leur disponibilité exemplaire et leurs réponses à nos questions par mail ou message, nous avons pu consolider nos connaissances et explorer de nouveaux domaines techniques.

Nous leur adressons également nos sincères remerciements pour nous avoir confié un sujet de projet tellement enrichissant. En plus de parfaire nos compétences, cela nous a grandement aidés lors de nos entretiens de recherche de stage, en nous permettant de mettre en avant une expérience significative.

Enfin, nous tenons à souligner et à saluer le temps et l'énergie investis par nos professeurs pour répondre à nos interrogations et clarifier les différentes problématiques rencontrées.

Table des matières

Table des figures	9
Introduction générale	11
1 Étude Préliminaire	12
1.1 Contexte Général	12
1.1.1 Contexte du Projet	12
1.1.2 Problématiques	12
1.1.3 Objectifs	13
1.2 Conclusion	14
2 Méthodologie et Phases de Développement	15
2.1 Conception des Diagrammes	15
2.1.1 Diagramme de Classes	15
2.1.2 Diagrammes de Séquence	16
2.2 Découpage en Microservices	18
2.2.1 Liste des Microservices	18
2.2.2 Communication entre Microservices	20
2.3 Intégration Continue et Déploiement Continu (CI/CD)	20
2.4 Gestion de la Qualité et Tests Automatisés	21
2.4.1 SonarQube	22
2.4.2 JMeter	22
2.4.3 Selenium	22
2.5 Planification et Gestion de Projet	23
2.6 Gestion des Versions et Collaboration	24

2.7	Conclusion	25
3	Organisation de l'Équipe et Répartition	27
3.1	Lecture et Reformulation du Cahier des Charges	27
3.2	Mise en Place des Diagrammes, Microservices et Corrections	27
3.3	Séparation des Microservices	27
3.4	Intégration de la Sécurité au Niveau de l'API Gateway	28
3.5	Conception du Front-End	28
3.6	Tests et Configuration Docker/CI-CD	28
3.7	Communication et Collaboration	28
3.8	Conclusion	29
4	Technologies Utilisées	30
4.1	Spring Boot	30
4.2	Angular avec ng-zorro	30
4.3	Kafka	31
4.4	SonarQube	32
4.5	JMeter	32
4.6	Selenium	32
4.7	Docker et GitHub Actions	33
4.8	MySQL	33
4.9	Jira	34
4.10	Postman	34
4.11	IntelliJ IDEA	34
4.12	StarUML	35
4.13	Jest	35
4.14	Maven	36
4.15	Twilio	36
4.16	phpMyAdmin	36
4.17	Spring Security	37
4.18	Conclusion	37

5	Captures d'écran de l'Application Front-End	38
5.1	Login	38
5.2	Réinitialisation du Mot de Passe	39
5.3	Barre Latérale Basée sur le Rôle	40
5.4	Gestion des Utilisateurs	43
5.5	Gestion des Laboratoires	43
5.6	Contacts des Laboratoires	44
5.7	Gestion des Patients	44
5.8	Gestion des Dossiers	45
5.9	Gestion des Examens	45
5.10	Gestion des Épreuves	46
5.11	Gestion des Tests D'épreuves	46
5.12	Gestion des Adresses	47
5.13	Consultation des Dossiers	47
5.14	Étapes dans les Laboratoires	48
6	Conclusion et Perspectives	49
	Bibliographie	50

Table des figures

2.1	Diagramme de Classes	16
2.2	Diagramme de Séquence : Technicien crée un dossier	17
2.3	Diagramme de Séquence : Technicien saisie resultats d'analyses	17
2.4	Diagramme de Séquence : Technicien vérifie et imprime un dossier	18
2.5	Architecture des Microservices	19
2.6	Structure du code back-end	19
2.7	Workflow GitHub Actions Sonarqube	21
2.8	Workflow GitHub Actions Testing and Pushing to DockerHub	21
2.9	Rapport de Tests SonarQube	23
2.10	Diagramme de Gantt du Projet	24
2.11	Repository GitHub	25
2.12	Gestion des Branches avec Gitflow	25
4.1	Logo de Spring Boot	30
4.2	Logo d'Angular	31
4.3	Logo de ng-zorro	31
4.4	Logo d'Apache Kafka	31
4.5	Logo de SonarQube	32
4.6	Logo de JMeter	32
4.7	Logo de Selenium	32
4.8	Logo de Docker	33
4.9	Logo de GitHub Actions	33
4.10	Logo de MySQL	33
4.11	Logo de Jira	34

4.12	Logo de Postman	34
4.13	Logo d'IntelliJ IDEA	35
4.14	Logo de StarUML	35
4.15	Logo de Jest	35
4.16	Logo de Maven	36
4.17	Logo de Twilio	36
4.18	Logo de phpMyAdmin	37
4.19	Logo de Spring Security	37
5.1	Page de Connexion	38
5.2	Page de Réinitialisation du Mot de Passe	39
5.3	Barre Latérale pour Administrateur	40
5.4	Barre Latérale pour Admin Labo	41
5.5	Barre Latérale pour Technicien	42
5.6	Page de Gestion des Utilisateurs	43
5.7	Page de Gestion des Laboratoires	43
5.8	Page des Contacts des Laboratoires	44
5.9	Page de Gestion des Patients	44
5.10	Page de Gestion des Dossiers	45
5.11	Page de Gestion des Examens	45
5.12	Page de Gestion des Épreuves	46
5.13	Page de Gestion des Tests D'épreuves	46
5.14	Page de Gestion des Adresses	47
5.15	Page de Consultation des Dossiers	47
5.16	Étapes dans les Laboratoires	48

Introduction générale

Dans un contexte où la gestion efficace des laboratoires est cruciale pour le bon fonctionnement des établissements de santé, il devient impératif de disposer de systèmes informatiques robustes et innovants. C'est dans cette optique que nous avons entrepris notre projet de fin d'études, réalisé au sein de notre deuxième année de cycle d'ingénieur en génie informatique à l'École Nationale des Sciences Appliquées de Khouribga.

Notre projet, intitulé *Système de Gestion de Laboratoire*, vise à développer une application complète permettant la gestion des utilisateurs, des patients, des dossiers médicaux, des laboratoires, et des analyses effectuées. Pour répondre aux exigences fonctionnelles et non fonctionnelles du cahier des charges, nous avons adopté une architecture microservices en utilisant Spring Boot pour le back-end et Angular avec ng-zorro pour le front-end.

La méthodologie suivie a débuté par une analyse approfondie du cahier des charges, suivie de la conception de diagrammes de classes et de séquence afin de structurer les différentes composantes du système. Nous avons ensuite implémenté les microservices nécessaires, tels que le Config Server, le Discovery Server, l'API Gateway sécurisée par JWT, ainsi que les services dédiés aux utilisateurs, aux patients, aux laboratoires et aux analyses. L'intégration de Kafka pour la messagerie asynchrone a permis d'assurer une communication efficace entre les microservices.

Côté front-end, l'utilisation d'Angular et de ng-zorro a facilité le développement d'une interface utilisateur intuitive et réactive, avec des fonctionnalités telles que des barres latérales repliables, des tables CRUD, des modales pour les opérations, et des pages de statistiques réservées aux utilisateurs autorisés. Pour garantir la qualité et la performance de notre application, nous avons intégré des outils tels que SonarQube pour l'analyse de la qualité du code, JMeter pour les tests de performance, et Selenium pour les tests automatisés de l'interface utilisateur.

La mise en place d'une pipeline CI/CD avec Docker et GitHub Actions a automatisé le processus de déploiement et de tests, assurant ainsi une livraison continue et fiable des fonctionnalités développées. Cette approche nous a non seulement permis de maintenir un haut niveau de qualité tout au long du développement, mais aussi de renforcer nos compétences en gestion de projet et en collaboration d'équipe.

Ce rapport se structure en plusieurs chapitres détaillant chacun une étape clé de notre projet, depuis la conception initiale jusqu'aux tests et à la mise en production. Il reflète notre engagement à développer une solution technique innovante répondant aux besoins réels des laboratoires tout en nous offrant une expérience enrichissante et formatrice dans le domaine du génie informatique.

Chapitre 1

Étude Préliminaire

1.1 Contexte Général

1.1.1 Contexte du Projet

Le développement technologique rapide et les exigences croissantes en matière de gestion efficace des laboratoires ont conduit à la nécessité de concevoir des systèmes informatiques robustes et innovants. Notre projet, intitulé *Système de Gestion de Laboratoire*, s'inscrit dans ce contexte en visant à fournir une solution complète et modulable pour la gestion des utilisateurs, des patients, des dossiers médicaux, des laboratoires et des analyses réalisées. Réalisé dans le cadre de notre deuxième année de cycle d'ingénieur en génie informatique à l'École Nationale des Sciences Appliquées de Khouribga, ce projet a pour objectif de répondre aux besoins fonctionnels et non fonctionnels définis dans le cahier des charges.

L'architecture adoptée repose sur les microservices avec Spring Boot pour le back-end et Angular avec ng-zorro pour le front-end. Cette approche permet une modularité accrue, facilitant la maintenance et l'évolution du système. De plus, l'intégration de technologies telles que Kafka pour la messagerie asynchrone, SonarQube pour l'analyse de la qualité du code, JMeter pour les tests de performance et Selenium pour les tests automatisés de l'interface utilisateur, renforce la fiabilité et la performance de l'application.

1.1.2 Problématiques

Lors de la conception du Système de Gestion de Laboratoire, plusieurs problématiques clés ont émergé, nécessitant des solutions adaptées pour garantir la réussite du projet. Ces problématiques sont essentielles pour assurer que le système réponde efficacement aux besoins des utilisateurs tout en étant performant et sécurisé.

- **Comment concevoir une architecture modulaire et évolutive qui facilite la maintenance et l'intégration de nouveaux services ?** L'adoption d'une architecture microservices vise à découper le système en services indépendants, permettant ainsi une évolution et une maintenance plus aisées.
- **Comment assurer la sécurité des données et des accès utilisateurs dans un environnement distribué ?** La mise en place de Spring Security avec JWT permet une gestion fine des rôles et des permissions, garantissant que seules les personnes autorisées

peuvent accéder à certaines fonctionnalités.

- **Comment garantir la performance et la fiabilité du système sous charge ?** L'utilisation de JMeter pour les tests de performance et l'intégration de Kafka pour la gestion des communications asynchrones assurent que le système peut gérer un grand nombre de requêtes tout en maintenant une performance optimale.
- **Comment automatiser les processus de déploiement et de tests pour assurer une livraison continue et de qualité ?** L'intégration de Docker et GitHub Actions dans la pipeline CI/CD permet d'automatiser les déploiements et les tests, réduisant ainsi les délais de livraison et augmentant la fiabilité des mises à jour.
- **Comment maintenir une haute qualité de code tout au long du développement ?** L'utilisation de SonarQube pour l'analyse continue de la qualité du code permet d'identifier et de corriger les problèmes avant qu'ils ne deviennent critiques, assurant ainsi un code propre et maintenable.

Ces problématiques guident notre démarche de développement, orientant les choix technologiques et méthodologiques pour répondre efficacement aux défis rencontrés.

1.1.3 Objectifs

Les objectifs de notre projet de Système de Gestion de Laboratoire sont multiples et visent à répondre aux besoins identifiés tout en assurant une solution robuste et adaptable. Ces objectifs sont articulés autour de plusieurs axes clés :

- **Développer une architecture microservices modulaire et scalable** : En structurant le système en services indépendants, nous visons à faciliter la maintenance, l'évolution et le déploiement des différentes composantes du système.
- **Assurer la sécurité et la gestion des accès** : Implémenter Spring Security avec JWT pour contrôler finement les accès aux différentes fonctionnalités, garantissant ainsi la protection des données sensibles et la conformité aux normes de sécurité.
- **Optimiser les performances du système** : Utiliser des outils comme JMeter pour effectuer des tests de charge et identifier les points de contention, permettant ainsi d'optimiser les performances et d'assurer une réactivité adéquate du système sous différentes charges d'utilisation.
- **Automatiser les processus de déploiement et de tests** : Mettre en place une pipeline CI/CD avec Docker et GitHub Actions pour automatiser les déploiements et les tests, réduisant ainsi les erreurs humaines et accélérant le cycle de développement.
- **Maintenir une haute qualité de code** : Intégrer SonarQube dans le processus de développement pour analyser en continu la qualité du code, facilitant ainsi l'identification et la résolution des problèmes et assurant un code maintenable et évolutif.
- **Faciliter la communication et la collaboration entre les services** : Utiliser Kafka pour gérer les communications asynchrones entre les microservices, améliorant ainsi la résilience et la scalabilité du système.
- **Améliorer l'expérience utilisateur** : Développer une interface front-end intuitive et réactive avec Angular et ng-zorro, incluant des fonctionnalités telles que des barres latérales repliables, des tables CRUD, des modales et des pages de statistiques réservées aux utilisateurs autorisés.
- **Valider l'interface utilisateur de manière automatisée** : Utiliser Selenium pour effectuer des tests automatisés de l'interface utilisateur, garantissant ainsi que les fonctionnalités front-end répondent aux attentes et fonctionnent correctement.

Ces objectifs structurent notre approche de développement et orientent les différentes phases du projet, depuis la conception jusqu'à la mise en production, en passant par les tests et l'intégration continue.

1.2 Conclusion

L'étude préliminaire de notre projet de Système de Gestion de Laboratoire a permis de définir clairement le contexte, d'identifier les problématiques majeures et de poser des objectifs précis pour guider le développement. En adoptant une architecture microservices et en intégrant des outils avancés tels que Spring Boot, Angular, Kafka, SonarQube, JMeter et Selenium, nous visons à créer un système performant, sécurisé et maintenable.

Cette analyse initiale a souligné l'importance de répondre aux exigences fonctionnelles et non fonctionnelles tout en tenant compte des défis technologiques et organisationnels. En structurant notre démarche autour des objectifs définis, nous nous assurons que chaque étape du projet contribue à la réalisation d'une solution complète et efficace, capable de répondre aux besoins des utilisateurs et de s'adapter aux évolutions futures.

Ainsi, cette phase d'étude préliminaire constitue une base solide pour le développement ultérieur du projet, en fournissant une vision claire des enjeux et en définissant les axes prioritaires pour atteindre les résultats escomptés.

Chapitre 2

Méthodologie et Phases de Développement

2.1 Conception des Diagrammes

La phase de conception des diagrammes a été cruciale pour structurer et visualiser l'architecture globale du système. Cette étape nous a permis de clarifier les relations entre les différentes entités et microservices, garantissant ainsi une conception robuste et évolutive.

2.1.1 Diagramme de Classes

Nous avons commencé par élaborer un diagramme de classes initial pour représenter toutes les entités du système. Ce diagramme a inclus les principales classes telles que **Utilisateur**, **Patient**, **Laboratoire**, **Analyse**, etc., ainsi que leurs relations et attributs essentiels.

- **Identification des Entités** : Chaque entité a été définie avec ses attributs et ses méthodes pertinentes. Par exemple, la classe **Utilisateur** comprend des attributs comme **id**, **nom**, **email**, **motDePasse**, et des méthodes pour la gestion des rôles et des permissions.
- **Relations entre Entités** : Les relations telles que les associations, les héritages et les compositions ont été établies pour refléter fidèlement les interactions entre les différentes entités. Par exemple, un **Laboratoire** peut être associé à plusieurs **Analyses**, tandis qu'un **Patient** peut avoir plusieurs **Dossiers**.

Cependant, lors de la validation initiale, nous avons identifié des relations circulaires entre certaines classes, ce qui rendait le diagramme difficile à maintenir et compréhensible. Pour remédier à cela, nous avons modifié le diagramme en réorganisant les relations et en introduisant des classes intermédiaires lorsque nécessaire.

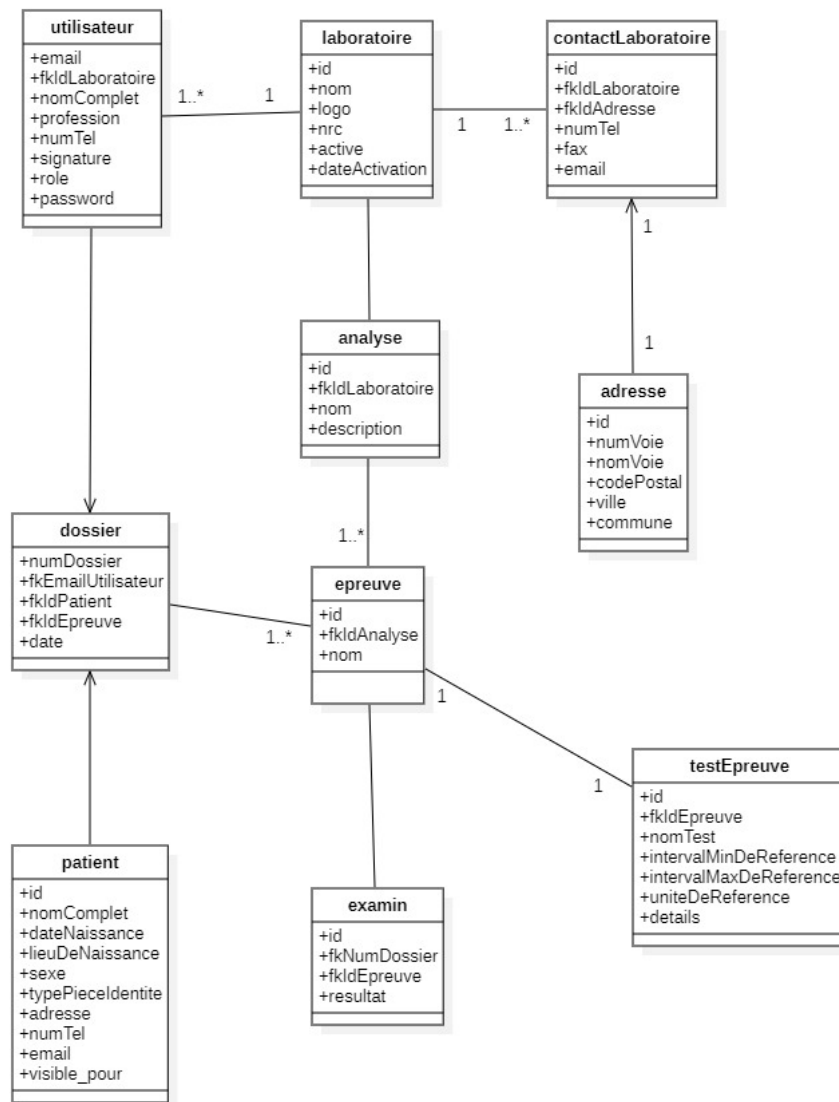


FIGURE 2.1 – Diagramme de Classes

2.1.2 Diagrammes de Séquence

Après avoir stabilisé le diagramme de classes, nous avons créé des diagrammes de séquence pour détailler les interactions entre les microservices lors des différentes opérations du système. Ces diagrammes ont été essentiels pour comprendre le flux de données et les appels de services nécessaires pour des fonctionnalités telles que la création d'un dossier patient, la gestion des analyses ou l'authentification des utilisateurs.

- **Flux de Création d'un Dossier** : Le diagramme de séquence illustre comment un utilisateur authentifié peut créer un dossier patient en interagissant avec le **Service Patient**, qui à son tour communique avec le **Service Utilisateur** pour vérifier les permissions.

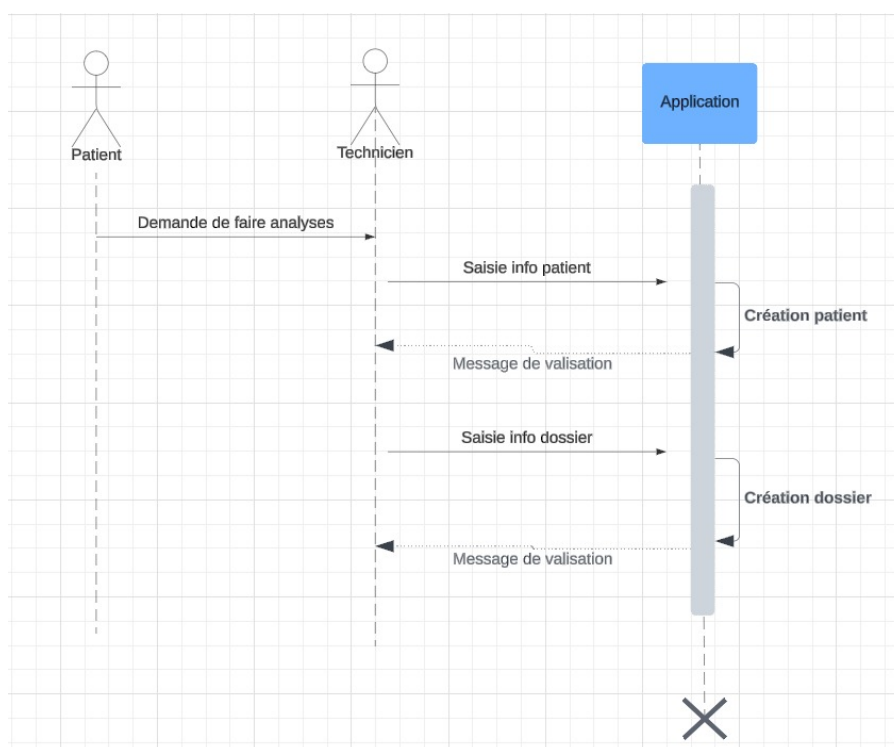


FIGURE 2.2 – Diagramme de Séquence : Technicien crée un dossier

- **Processus d'Analyse** : Ce diagramme montre comment une demande d'analyse est traitée par le **Service Analyse**, en communiquant avec le **Service Laboratoire** pour enregistrer les résultats et notifier les utilisateurs concernés via le **Service de Messagerie**.

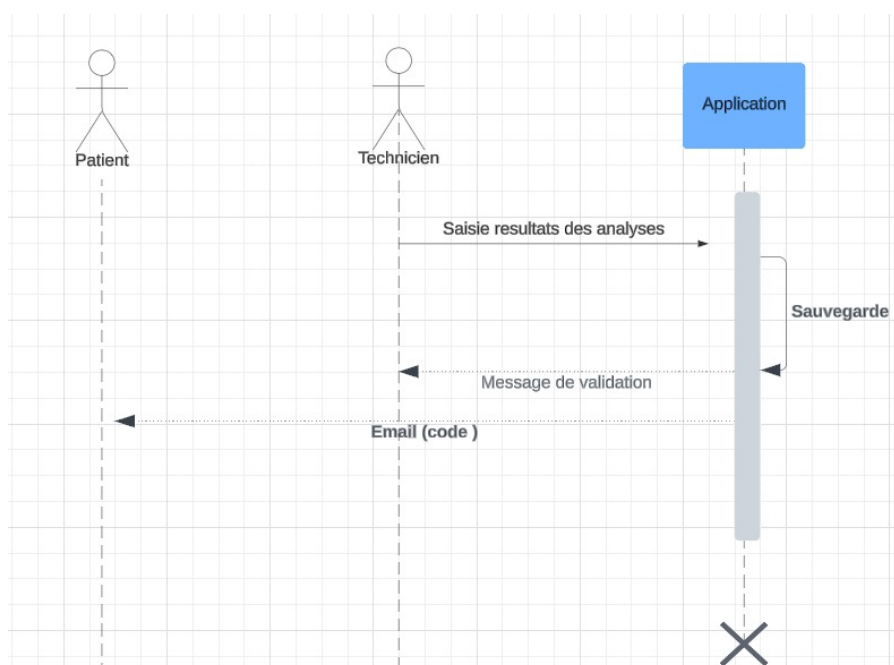


FIGURE 2.3 – Diagramme de Séquence : Technicien saisie resultats d'analyses

- **Vérification et Impression d'un Dossier** : Ce diagramme de séquence illustre le processus par lequel un technicien vérifie un dossier patient et imprime les résultats. Le technicien envoie une requête au **Service Patient** pour accéder aux informations du dossier. Une fois les données récupérées, le technicien peut initier l'impression des résultats via le **Service d'Impression**, qui communique avec l'imprimante configurée.

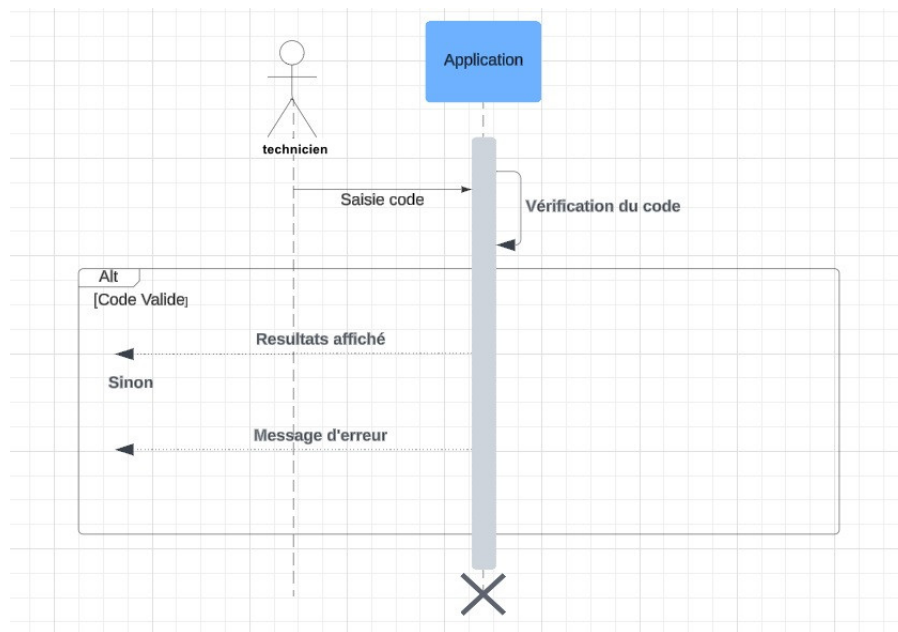


FIGURE 2.4 – Diagramme de Séquence : Technicien vérifie et imprime un dossier

Ces diagrammes de séquence ont non seulement aidé à clarifier les interactions entre les services, mais ont également servi de base pour le développement et les tests, assurant ainsi que chaque fonctionnalité est correctement implémentée et intégrée.

2.2 Découpage en Microservices

L'architecture microservices a été choisie pour sa modularité, sa scalabilité et sa facilité de maintenance. Chaque microservice est conçu pour être indépendant, ce qui permet de développer, déployer et scaler chaque composant de manière autonome.

2.2.1 Liste des Microservices

Nous avons développé plusieurs microservices, chacun ayant une responsabilité spécifique au sein du système :

- **Config Server** : Ce microservice centralise les configurations de tous les autres microservices, facilitant ainsi la gestion et la mise à jour des paramètres de configuration de manière centralisée.
- **Discovery Server (Eureka)** : Utilisé pour l'enregistrement et la découverte dynamique des microservices, il permet à chaque service de s'annoncer et de localiser les autres services nécessaires à son fonctionnement.
- **API Gateway** : Servant de point d'entrée principal pour les requêtes externes, l'API Gateway gère l'authentification via Spring Security et JWT, assure le routage des requêtes vers les microservices appropriés, et implémente le contrôle d'accès basé sur les rôles des utilisateurs.
- **Service Utilisateur** : Responsable de la gestion des utilisateurs, ce service gère la création, la mise à jour des profils, l'archivage des comptes inactifs, ainsi que les fonctionnalités d'authentification et de réinitialisation des mots de passe.

- **Service Patient** : Ce service gère les informations relatives aux patients, incluant la création et la mise à jour des dossiers, la gestion des examens et l'historique médical.
- **Service Laboratoire** : Chargé de la gestion des laboratoires, des adresses associées, et des contacts, ce service assure également la liaison avec les analyses réalisées.
- **Service Analyse** : Responsable de la gestion des analyses effectuées, des épreuves associées et des tests réalisés sur les échantillons.
- **Service de Messagerie (Kafka)** : Utilisé pour la communication asynchrone entre les microservices, Kafka permet l'envoi et la réception d'événements tels que les notifications ou les mises à jour de données critiques.

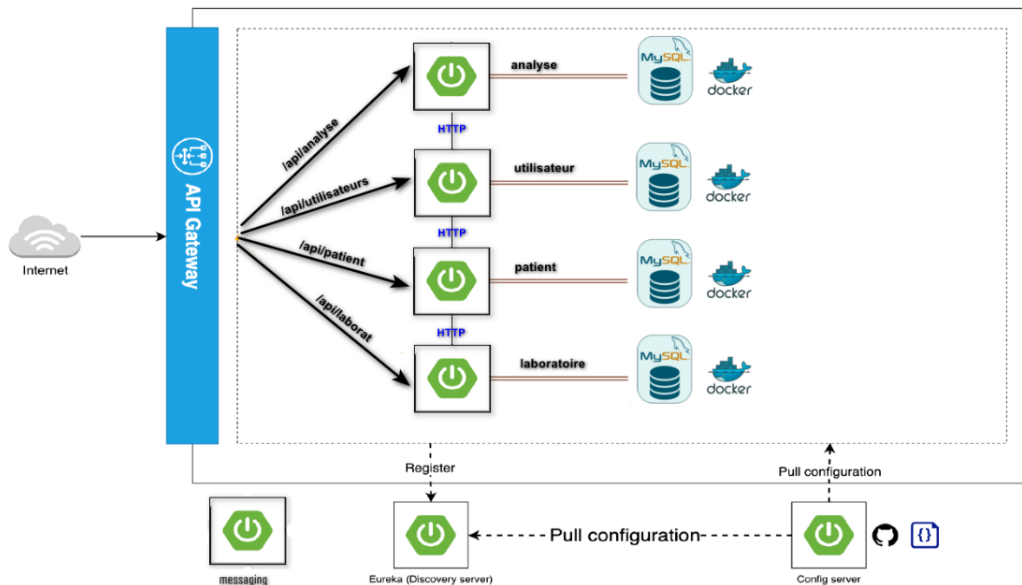


FIGURE 2.5 – Architecture des Microservices

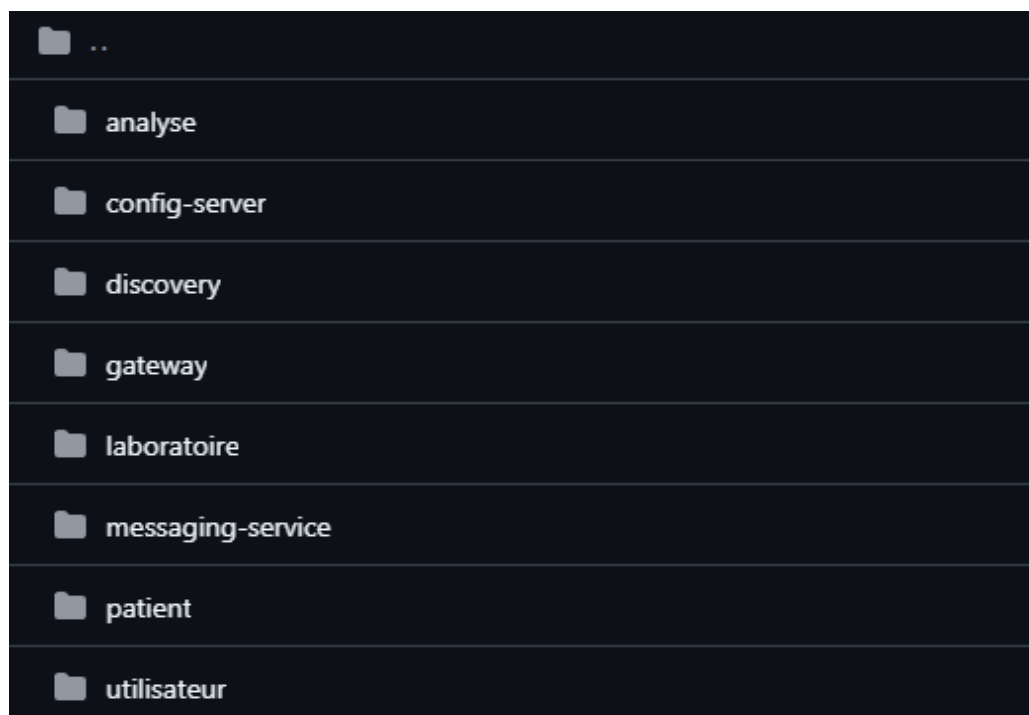


FIGURE 2.6 – Structure du code back-end

2.2.2 Communication entre Microservices

Tous les microservices communiquent principalement via des APIs REST, facilitant ainsi les interactions et l'échange de données. Dans certains cas, nous avons utilisé des clients Feign pour simplifier les appels entre services, en automatisant la gestion des requêtes HTTP et en intégrant des mécanismes de tolérance aux pannes.

Le stockage des données est centralisé dans une base de données MySQL, assurant une persistance fiable et cohérente des informations. Chaque microservice dispose de son propre schéma de base de données pour éviter les dépendances et favoriser l'indépendance des services.

2.3 Intégration Continue et Déploiement Continu (CI/CD)

Pour assurer une livraison rapide et fiable des fonctionnalités, nous avons mis en place une pipeline CI/CD automatisée. Cette pipeline intègre plusieurs outils permettant de tester, analyser et déployer les microservices de manière cohérente.

- **Docker et Docker Compose** : Chaque microservice possède son propre Dockerfile, permettant de créer des conteneurs isolés et reproductibles. Docker Compose a été utilisé pour orchestrer ces conteneurs localement, facilitant le développement et les tests en environnement simili-production.
- **GitHub Actions** : Nous avons utilisé GitHub Actions pour automatiser les tests unitaires et d'intégration à chaque commit, exploitant les ressources cloud de GitHub pour accélérer le processus. Ces workflows vérifient la qualité du code, exécutent les tests et construisent les images Docker nécessaires au déploiement.
- **SonarQube** : Intégré dans la pipeline CI/CD via un runner local, SonarQube analyse la qualité du code en continu, identifiant les duplications, les bugs potentiels, et mesurant la couverture des tests. Cela assure un maintien constant des standards de qualité tout au long du développement.
- **Docker Hub** : Après la réussite des tests, les images Docker sont automatiquement poussées vers Docker Hub, un registre de conteneurs, facilitant leur déploiement sur différents environnements (développement, test, production).

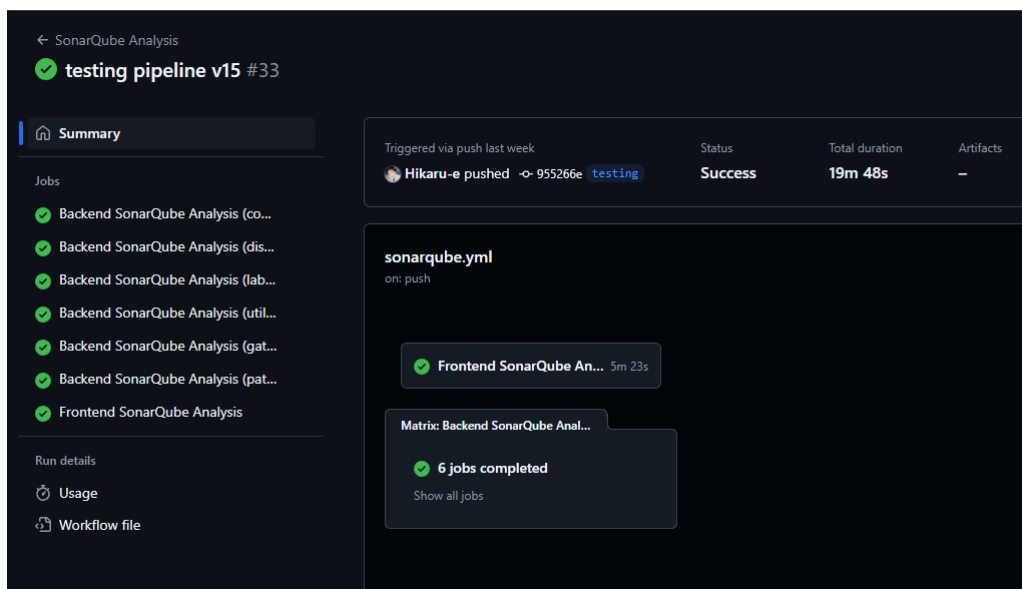


FIGURE 2.7 – Workflow GitHub Actions Sonarqube

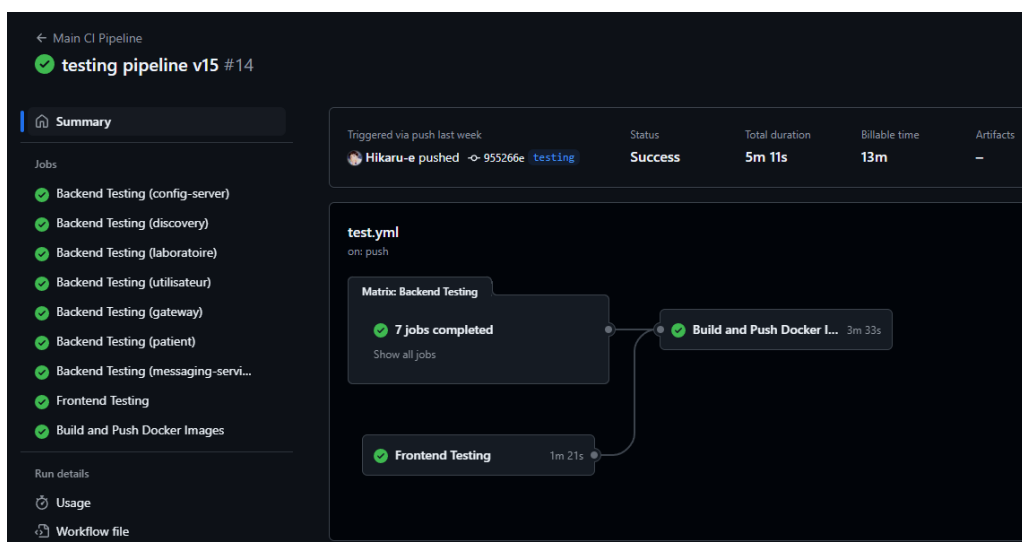


FIGURE 2.8 – Workflow GitHub Actions Testing and Pushing to DockerHub

Cette approche CI/CD a non seulement automatisé les processus de test et de déploiement, mais a également amélioré la collaboration au sein de l'équipe en réduisant les risques d'erreurs humaines et en accélérant le cycle de développement.

2.4 Gestion de la Qualité et Tests Automatisés

Pour garantir la fiabilité et la robustesse du système, nous avons intégré divers outils de gestion de la qualité et de tests automatisés dans notre processus de développement.

2.4.1 SonarQube

SonarQube a été utilisé pour effectuer une analyse continue de la qualité du code. Il aide à identifier les problèmes tels que les duplications de code, les bugs potentiels, les vulnérabilités de sécurité, et évalue la couverture des tests. En intégrant SonarQube dans notre pipeline CI/CD, nous avons pu maintenir un niveau élevé de qualité tout au long du cycle de développement.

2.4.2 JMeter

JMeter a été employé pour réaliser des tests de performance et de charge sur le système. Ces tests nous ont permis de mesurer la capacité du système à gérer un grand nombre de requêtes simultanées, d'identifier les goulets d'étranglement et d'optimiser les performances des microservices en conséquence.

2.4.3 Selenium

Selenium a été utilisé pour automatiser les tests de l'interface utilisateur. En créant des scripts de tests automatisés, nous avons pu vérifier que les interactions côté front-end fonctionnent comme prévu, assurant ainsi une expérience utilisateur sans faille et réduisant le risque de régressions lors des mises à jour.

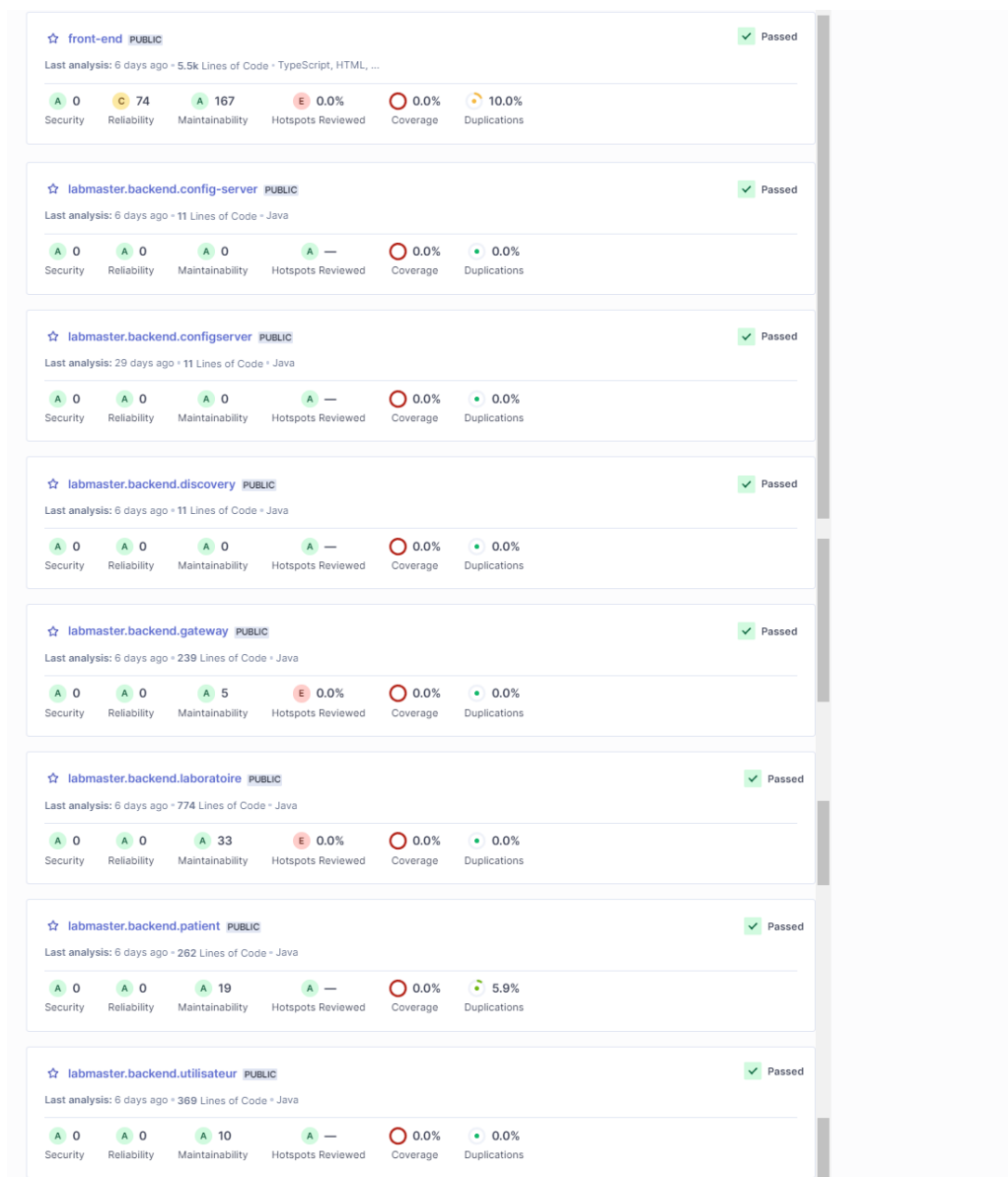


FIGURE 2.9 – Rapport de Tests SonarQube

Ces outils ont joué un rôle essentiel dans la garantie de la qualité du système, en automatisant la détection des problèmes et en facilitant les itérations rapides durant le développement.

2.5 Planification et Gestion de Projet

Une gestion efficace du projet a été essentielle pour respecter les délais et garantir la qualité des livrables. Nous avons adopté des méthodologies agiles, telles que Scrum, pour structurer notre travail en sprints itératifs.

- **Sprints** : Chaque sprint de deux semaines a été dédié à des objectifs précis, incluant le développement de nouvelles fonctionnalités, la correction de bugs, et l'amélioration continue du système.
- **Réunions Quotidiennes** : Des réunions stand-up quotidiennes ont permis de suivre l'avancement du projet, d'identifier les obstacles et de coordonner les efforts de l'équipe.

- **Outils de Gestion** : Nous avons utilisé des outils comme Jira pour le suivi des tâches, Trello pour la gestion visuelle des projets, et GitHub pour la gestion du code source et la collaboration.
- **Revues de Code** : Des revues de code régulières ont été effectuées pour assurer le respect des standards de codage, partager les connaissances au sein de l'équipe et détecter les problèmes potentiels avant qu'ils ne se propagent.

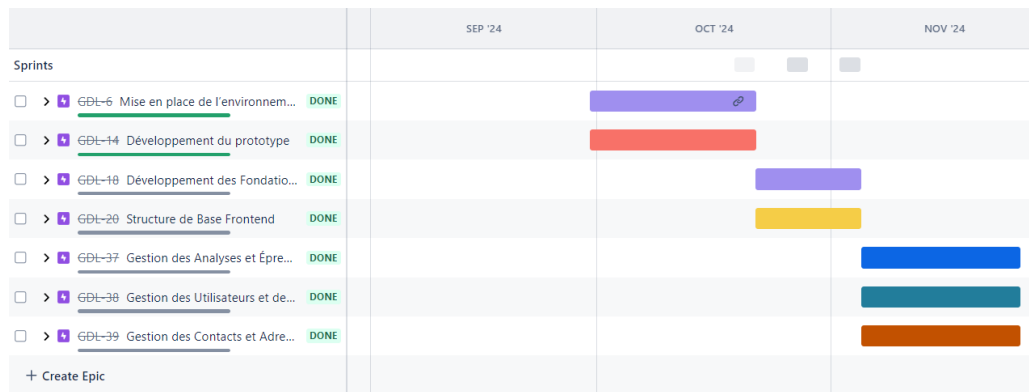


FIGURE 2.10 – Diagramme de Gantt du Projet

Cette approche structurée a permis une gestion transparente et efficace du projet, favorisant la collaboration et l'adaptabilité face aux changements et aux imprévus.

2.6 Gestion des Versions et Collaboration

La gestion rigoureuse des versions et la collaboration efficace au sein de l'équipe ont été soutenues par l'utilisation de Git et GitHub.

- **Git** : Utilisé pour le versionnage du code, Git nous a permis de suivre les modifications, gérer les branches de développement et fusionner les contributions de manière contrôlée.
- **GitHub** : En plus de servir de dépôt centralisé, GitHub a facilité la collaboration grâce aux pull requests, aux revues de code et aux discussions autour des issues. L'intégration avec GitHub Actions a également permis d'automatiser les workflows CI/CD.
- **Branches** : Nous avons adopté une stratégie de branches Gitflow, avec des branches dédiées pour le développement, les tests et les déploiements en production, assurant ainsi une gestion claire et ordonnée des différentes phases du cycle de vie du projet.

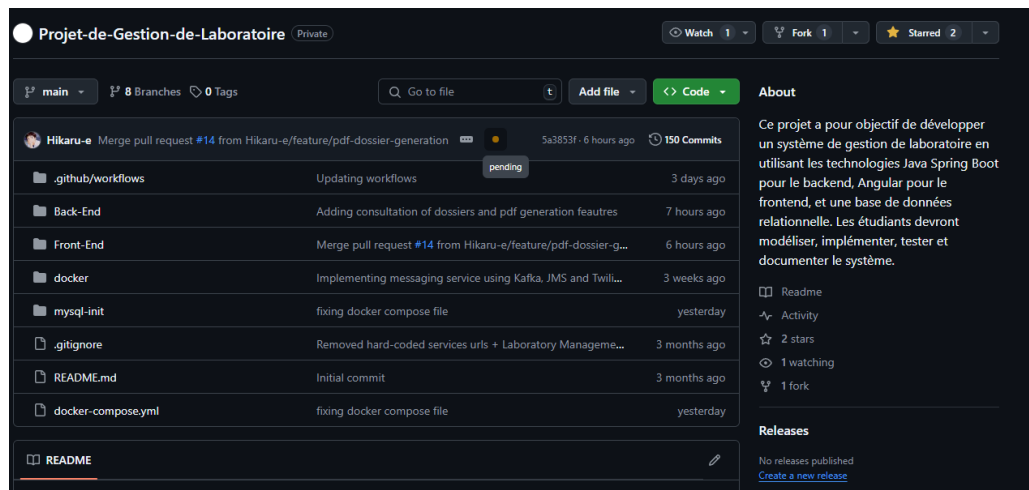


FIGURE 2.11 – Repository GitHub

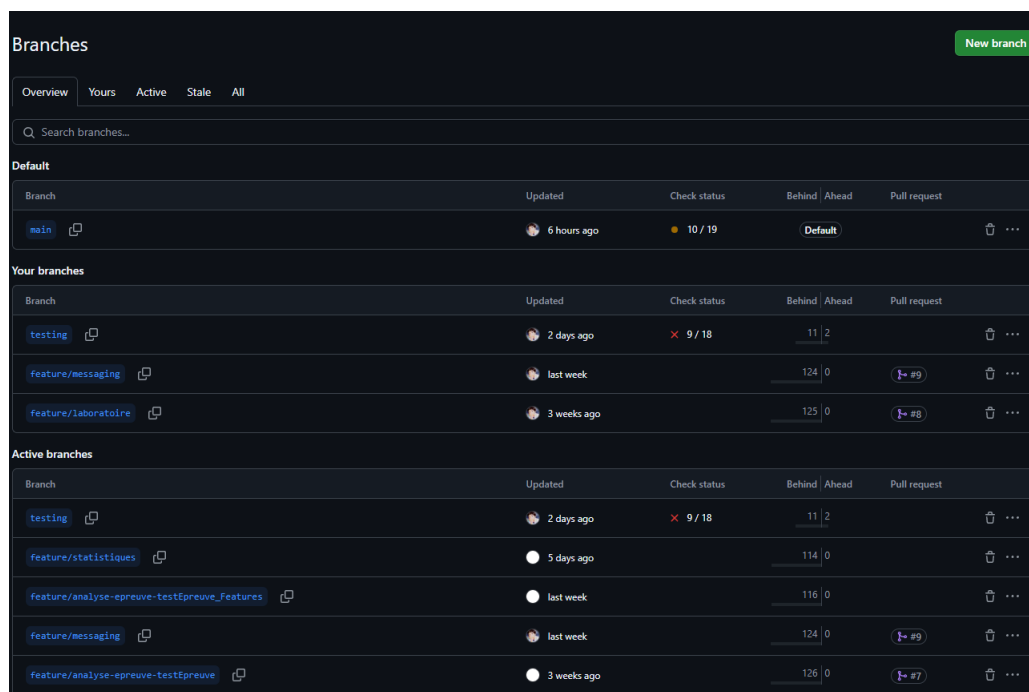


FIGURE 2.12 – Gestion des Branches avec Gitflow

Cette gestion des versions et cette collaboration via GitHub ont renforcé l'efficacité de l'équipe, facilitant le partage des connaissances et la cohérence du code tout au long du projet.

2.7 Conclusion

La méthodologie adoptée et les phases de développement mises en œuvre ont été déterminantes pour la réussite de notre projet de Système de Gestion de Laboratoire. En commençant par une conception rigoureuse des diagrammes, nous avons pu établir une architecture solide et bien structurée, facilitant le découpage en microservices indépendants mais interconnectés.

Le choix des technologies telles que Spring Boot, Angular avec ng-zorro, Kafka, SonarQube, JMeter, et Selenium a permis de répondre efficacement aux exigences de performance, de sécurité et de qualité du projet. L'intégration de la pipeline CI/CD avec Docker et GitHub Actions

a automatisé les processus critiques, assurant une livraison continue et fiable des fonctionnalités développées.

La gestion de projet agile, appuyée par des outils de gestion comme Jira et Trello, ainsi que les pratiques de gestion des versions via Git et GitHub, ont favorisé une collaboration efficace et une adaptation rapide aux changements et aux défis rencontrés.

Enfin, l'utilisation d'outils de gestion de la qualité et de tests automatisés a garanti que le système développé soit robuste, performant et maintenable, répondant ainsi aux besoins des utilisateurs tout en respectant les standards de l'industrie.

Cette méthodologie structurée et l'approche itérative ont non seulement conduit à la réalisation d'un système performant et évolutif, mais ont également renforcé nos compétences en développement logiciel, gestion de projet, et collaboration d'équipe. Le succès de ce projet témoigne de l'efficacité des stratégies mises en place et sert de fondation solide pour nos futurs développements et projets professionnels.

Chapitre 3

Organisation de l'Équipe et Répartition

La réussite de notre projet de Système de Gestion de Laboratoire repose en grande partie sur une organisation efficace de l'équipe et une répartition claire des tâches. Afin d'assurer une collaboration harmonieuse et une productivité optimale, nous avons mis en place une structure de travail bien définie, favorisant la complémentarité des compétences et la communication continue entre les membres de l'équipe.

3.1 Lecture et Reformulation du Cahier des Charges

La première étape cruciale de notre projet a consisté en la lecture approfondie et la reformulation du cahier des charges. Cette phase a été réalisée de manière collaborative, impliquant tous les membres de l'équipe. Ensemble, nous avons analysé les besoins fonctionnels et non fonctionnels, identifié les exigences clés, et clarifié les objectifs du projet. Cette approche conjointe a permis de s'assurer que chaque membre comprenait parfaitement les attentes et les contraintes, facilitant ainsi une planification cohérente et une mise en œuvre efficace des différentes tâches à venir.

3.2 Mise en Place des Diagrammes, Microservices et Corrections

Une fois le cahier des charges bien compris, nous avons entamé la phase de conception, qui a inclus la création des diagrammes de classe et de séquence. Cette étape a nécessité une concertation régulière au sein de l'équipe afin de s'assurer que chaque diagramme reflétait fidèlement l'architecture envisagée. Les diagrammes ont servi de guides visuels indispensables pour structurer les microservices et définir les interactions entre eux. Lors de cette phase, nous avons également identifié et corrigé des incohérences ou des relations circulaires dans les diagrammes, garantissant ainsi une conception robuste et évolutive.

3.3 Séparation des Microservices

Pour optimiser la modularité et la scalabilité de notre système, nous avons procédé à la séparation des microservices. Chaque membre de l'équipe s'est vu assigner un ou deux microservices spécifiques, en fonction de ses compétences et de ses affinités. Cette répartition permet une res-

ponsabilité claire et une spécialisation dans le développement de chaque composant du système. Par exemple, certains membres ont été chargés du développement des services Utilisateur et Patient, tandis que d'autres ont pris en main les services Laboratoire et Analyse. Cette approche a favorisé une progression parallèle et efficace du projet, tout en maintenant une cohésion globale entre les différentes parties du système.

3.4 Intégration de la Sécurité au Niveau de l'API Gateway

L'intégration de la sécurité constitue une étape essentielle pour garantir la protection des données et des accès au système. Cette tâche a été réalisée de manière coordonnée, impliquant une collaboration étroite entre les développeurs backend et les spécialistes en sécurité. Nous avons mis en place Spring Security avec JWT (JSON Web Tokens) au niveau de l'API Gateway, assurant ainsi une authentification robuste et une gestion fine des rôles et permissions des utilisateurs. Cette intégration a nécessité des efforts conjoints pour configurer correctement les différents paramètres de sécurité et tester les mécanismes d'authentification, garantissant ainsi une protection efficace contre les accès non autorisés.

3.5 Conception du Front-End

Le développement du front-end a été réparti de manière à exploiter au mieux les compétences de chaque membre de l'équipe. Chaque développeur a été responsable d'un ou deux modules Angular spécifiques, en utilisant le framework ng-zorro pour assurer une interface utilisateur cohérente et réactive. Par exemple, certains membres ont travaillé sur la gestion des laboratoires et des analyses, tandis que d'autres se sont concentrés sur les interfaces de gestion des utilisateurs et des patients. Cette répartition permet une spécialisation et une accélération du développement, tout en facilitant l'intégration des différents modules pour créer une application front-end harmonieuse et fonctionnelle.

3.6 Tests et Configuration Docker/CI-CD

Les phases de tests et de configuration de l'infrastructure CI/CD ont été réparties selon les affinités et les compétences de chaque membre de l'équipe. Certains se sont spécialisés dans l'écriture des scripts de tests automatisés en utilisant des outils comme JUnit, Mockito, Jest et Selenium, tandis que d'autres ont pris en charge la configuration des conteneurs Docker et l'intégration des pipelines CI/CD avec GitHub Actions. Cette répartition a permis une couverture complète des besoins en matière de tests et de déploiement, assurant ainsi la qualité et la fiabilité du système tout au long du cycle de développement.

3.7 Communication et Collaboration

Tout au long du projet, une communication fluide et régulière a été maintenue grâce à l'utilisation d'outils de collaboration tels que Discord pour les discussions quotidiennes, Jira pour le

suivi des tâches, et GitHub pour la gestion du code source et les revues de code. Des réunions hebdomadaires ont été organisées pour évaluer l'avancement du projet, discuter des problèmes rencontrés et ajuster les plans de travail en conséquence. Cette organisation a favorisé une coordination efficace, une transparence dans les actions de chaque membre, et une résolution rapide des obstacles, permettant ainsi de maintenir le projet sur la bonne voie.

3.8 Conclusion

L'organisation méthodique de l'équipe et la répartition stratégique des tâches ont été déterminantes pour la progression harmonieuse de notre projet de Système de Gestion de Laboratoire. En collaborant étroitement dès la phase initiale de lecture du cahier des charges, en structurant efficacement les microservices, et en intégrant la sécurité de manière coordonnée, nous avons pu développer un système robuste et modulable. La division du travail en fonction des compétences et des affinités a optimisé notre productivité, tandis que l'utilisation d'outils de gestion de projet et de communication a assuré une collaboration fluide et efficace. Cette organisation a non seulement contribué à la réussite technique du projet, mais a également renforcé nos compétences en travail d'équipe et en gestion de projet, préparant ainsi le terrain pour des développements futurs et des projets professionnels ambitieux.

Chapitre 4

Technologies Utilisées

Dans le développement de notre Système de Gestion de Laboratoire, nous avons utilisé une variété de technologies modernes et éprouvées. Chaque technologie a été choisie pour ses avantages spécifiques en termes de performance, de scalabilité, de facilité de développement, et de support communautaire.

4.1 Spring Boot

Spring Boot a été le cœur de notre backend, facilitant la création de microservices robustes et autonomes. Sa capacité à simplifier la configuration et à intégrer facilement d'autres composants Spring comme Spring Security et Spring Cloud a été essentielle pour notre architecture.



FIGURE 4.1 – Logo de Spring Boot

4.2 Angular avec ng-zorro

Pour le frontend, nous avons utilisé Angular en combinaison avec ng-zorro, une bibliothèque de composants basée sur Ant Design. Cette combinaison nous a permis de développer rapidement une interface utilisateur réactive et esthétiquement agréable.



FIGURE 4.2 – Logo d'Angular



FIGURE 4.3 – Logo de ng-zorro

4.3 Kafka

Apache Kafka a été utilisé pour gérer la communication asynchrone entre les microservices. Sa capacité à traiter de grands volumes de messages en temps réel a été cruciale pour la réactivité et la résilience de notre système.



FIGURE 4.4 – Logo d'Apache Kafka

4.4 SonarQube

SonarQube a été intégré pour assurer une analyse continue de la qualité du code. Il nous a aidés à identifier et corriger les duplications, les bugs potentiels, et les vulnérabilités de sécurité, garantissant ainsi un code propre et maintenable.



FIGURE 4.5 – Logo de SonarQube

4.5 JMeter

JMeter a été employé pour réaliser des tests de performance et de charge. Ces tests nous ont permis de mesurer la capacité de notre système à gérer de nombreux utilisateurs simultanés et de détecter les goulots d'étranglement potentiels.



FIGURE 4.6 – Logo de JMeter

4.6 Selenium

Selenium a été utilisé pour automatiser les tests de l'interface utilisateur. En créant des scripts de tests, nous avons pu vérifier que les interactions utilisateurs fonctionnaient comme prévu, assurant ainsi une expérience utilisateur fluide et sans faille.



FIGURE 4.7 – Logo de Selenium

4.7 Docker et GitHub Actions

Docker a été utilisé pour conteneuriser nos applications, assurant une portabilité et une reproductibilité des environnements de développement et de production. GitHub Actions a été intégré dans notre pipeline CI/CD pour automatiser le déploiement et les tests, garantissant ainsi une livraison continue et fiable des fonctionnalités développées.

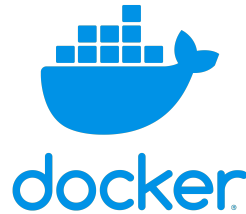


FIGURE 4.8 – Logo de Docker



FIGURE 4.9 – Logo de GitHub Actions

4.8 MySQL

MySQL a été choisi comme système de gestion de base de données relationnelle pour sa fiabilité, sa performance, et sa facilité d'intégration avec les microservices Spring Boot.



FIGURE 4.10 – Logo de MySQL

4.9 Jira

Jira est un outil de gestion de projet et de suivi des bugs largement utilisé pour sa flexibilité, ses fonctionnalités d'intégration, et sa capacité à gérer des projets Agile efficacement.



FIGURE 4.11 – Logo de Jira

4.10 Postman

Postman est une plateforme d'API qui facilite les tests, le développement, et la documentation des API, assurant une meilleure collaboration entre les développeurs.



FIGURE 4.12 – Logo de Postman

4.11 IntelliJ IDEA

IntelliJ IDEA est un environnement de développement intégré (IDE) reconnu pour sa puissance, sa prise en charge des frameworks modernes, et sa capacité à augmenter la productivité des développeurs.



FIGURE 4.13 – Logo d’IntelliJ IDEA

4.12 StarUML

StarUML est un outil de modélisation UML populaire qui permet de créer des diagrammes pour la conception de logiciels, facilitant une meilleure compréhension des systèmes.



FIGURE 4.14 – Logo de StarUML

4.13 Jest

Jest est un framework de test JavaScript moderne conçu pour assurer la fiabilité du code avec des tests automatisés rapides et efficaces.

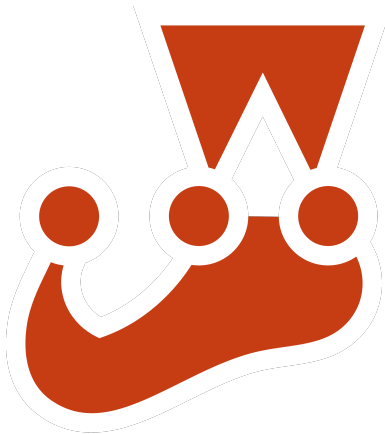


FIGURE 4.15 – Logo de Jest

4.14 Maven

Maven est un outil de gestion de projet et de compréhension des dépendances, très apprécié dans les projets Java pour son efficacité dans la gestion du cycle de vie.



FIGURE 4.16 – Logo de Maven

4.15 Twilio

Twilio est une plateforme cloud qui permet d'intégrer facilement des fonctionnalités de communication telles que les SMS, appels vocaux et vidéos dans les applications.



FIGURE 4.17 – Logo de Twilio

4.16 phpMyAdmin

phpMyAdmin est un outil basé sur le web qui permet de gérer facilement les bases de données MySQL et MariaDB grâce à une interface utilisateur intuitive.



FIGURE 4.18 – Logo de phpMyAdmin

4.17 Spring Security

Spring Security est un framework puissant qui offre des fonctionnalités de sécurité pour les applications Spring Boot, notamment l'authentification et l'autorisation.



FIGURE 4.19 – Logo de Spring Security

4.18 Conclusion

L'utilisation de ces technologies a permis de développer un système robuste, scalable et maintenable. Chaque outil et framework a été choisi pour ses avantages spécifiques, contribuant ainsi à la réussite globale du projet. En intégrant ces technologies modernes, nous avons non seulement répondu aux exigences du cahier des charges, mais avons également renforcé notre expertise technique dans des domaines clés du développement logiciel moderne.

Chapitre 5

Captures d'écran de l'Application Front-End

Dans cette section, nous présentons des captures d'écran de l'interface utilisateur de notre application front-end développée avec Angular et ng-zorro. Ces captures illustrent les différentes fonctionnalités et l'ergonomie de l'application, démontrant l'efficacité de notre conception et le respect des exigences fonctionnelles.

5.1 Login

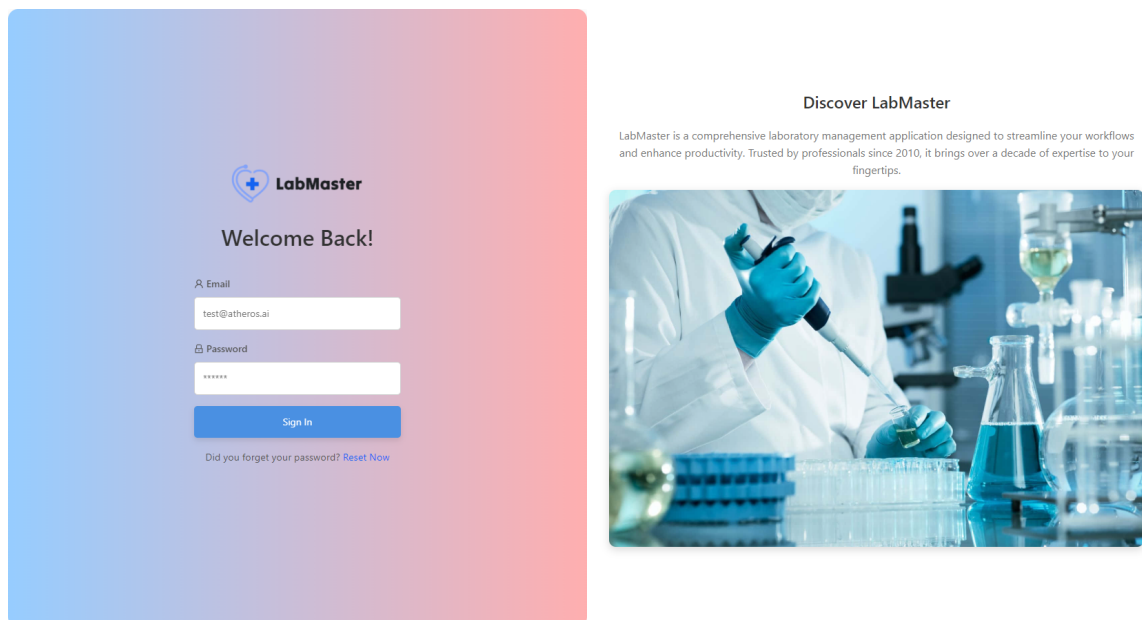


FIGURE 5.1 – Page de Connexion

5.2 Réinitialisation du Mot de Passe

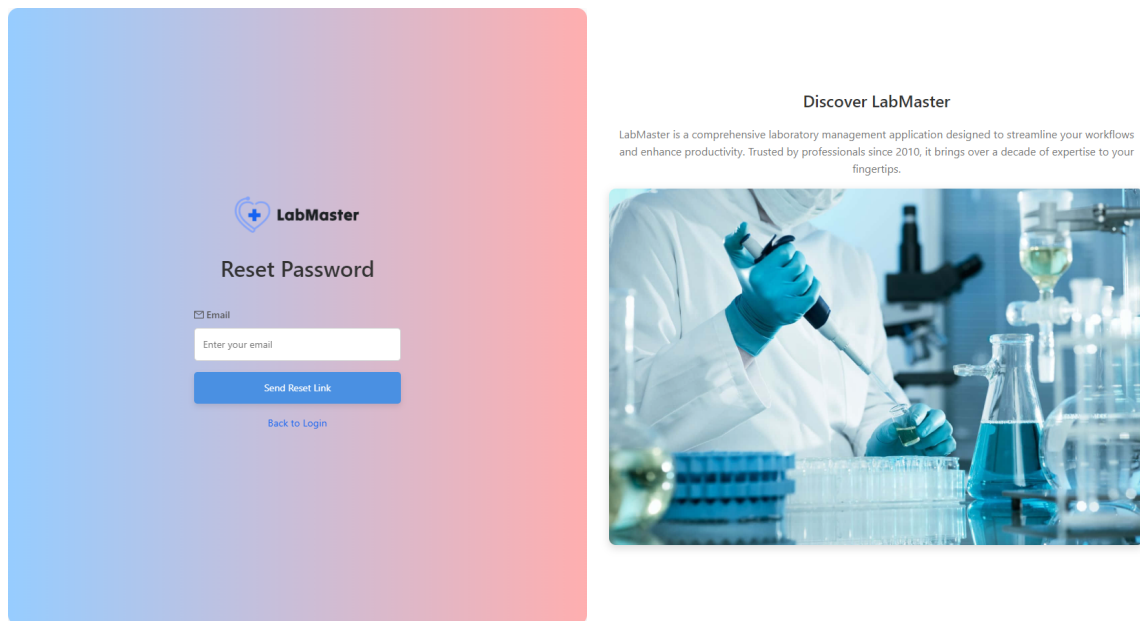


FIGURE 5.2 – Page de Réinitialisation du Mot de Passe

5.3 Barre Latérale Basée sur le Rôle

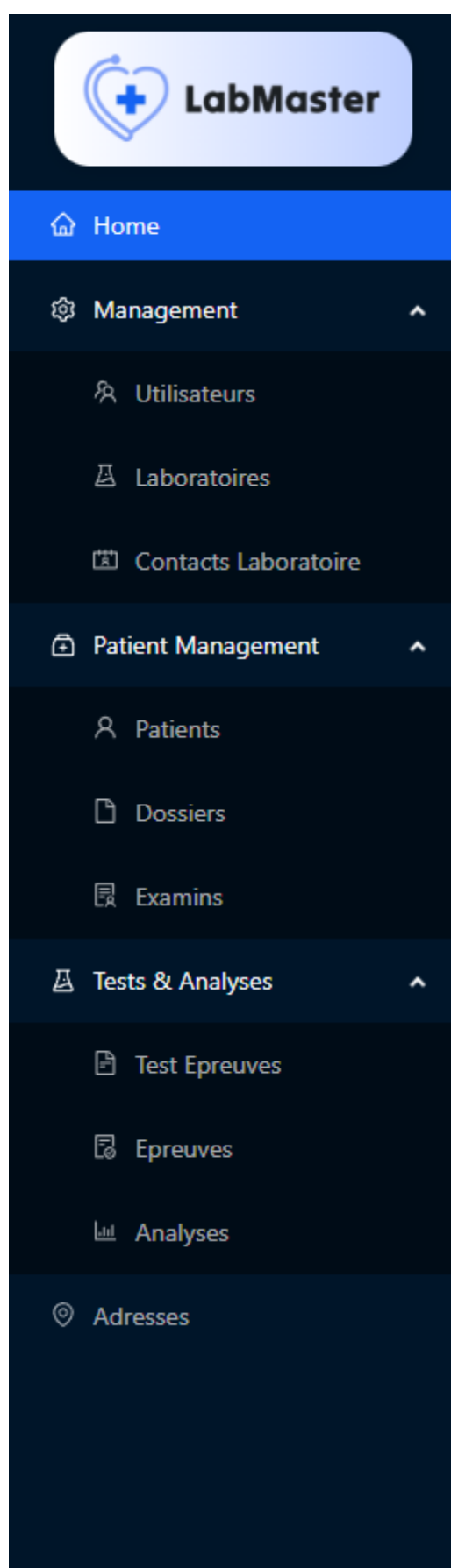


FIGURE 5.3 – Barre Latérale pour Administrateur

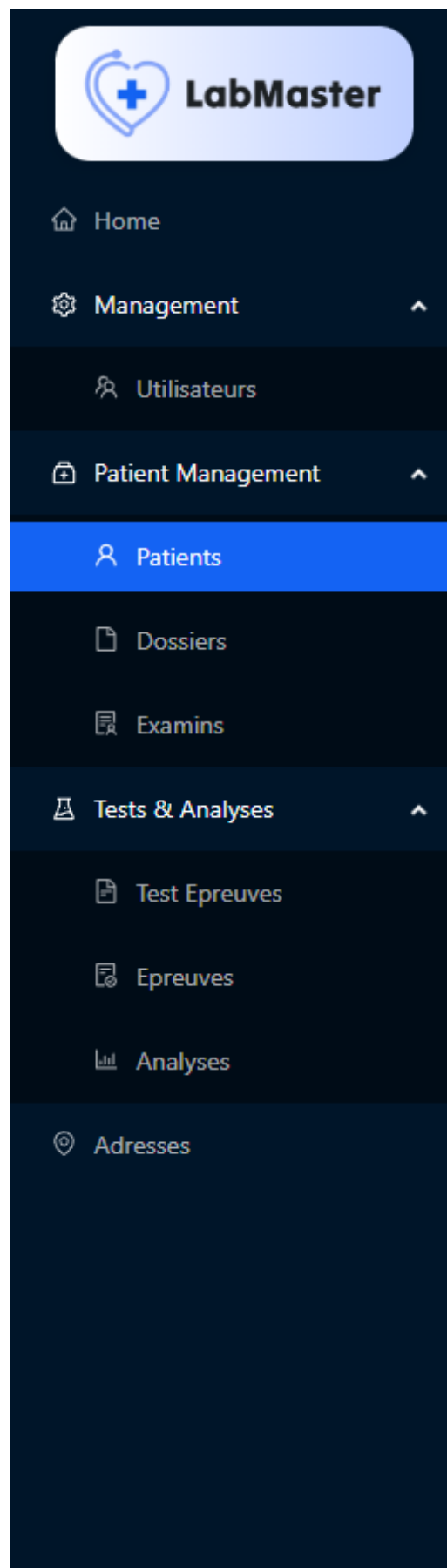


FIGURE 5.4 – Barre Latérale pour Admin Labo

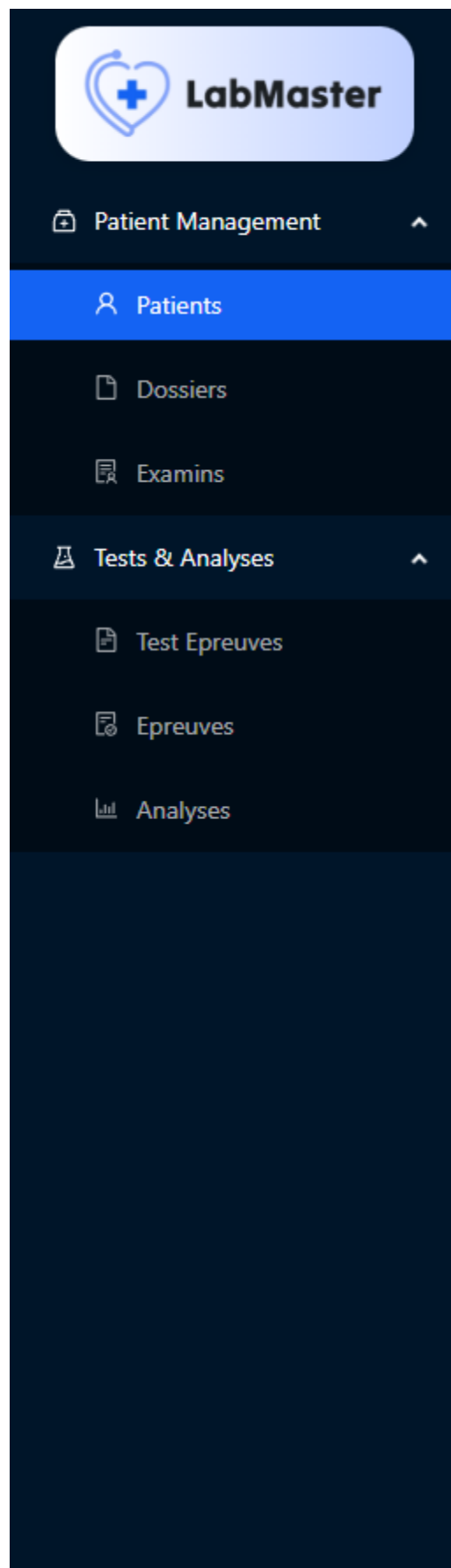


FIGURE 5.5 – Barre Latérale pour Technicien

5.4 Gestion des Utilisateurs

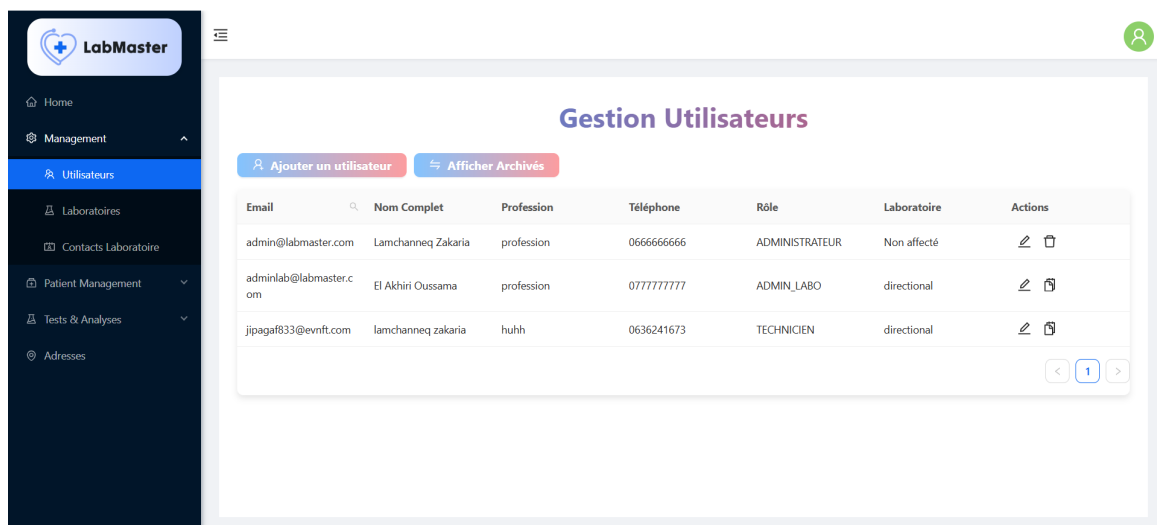


FIGURE 5.6 – Page de Gestion des Utilisateurs

5.5 Gestion des Laboratoires

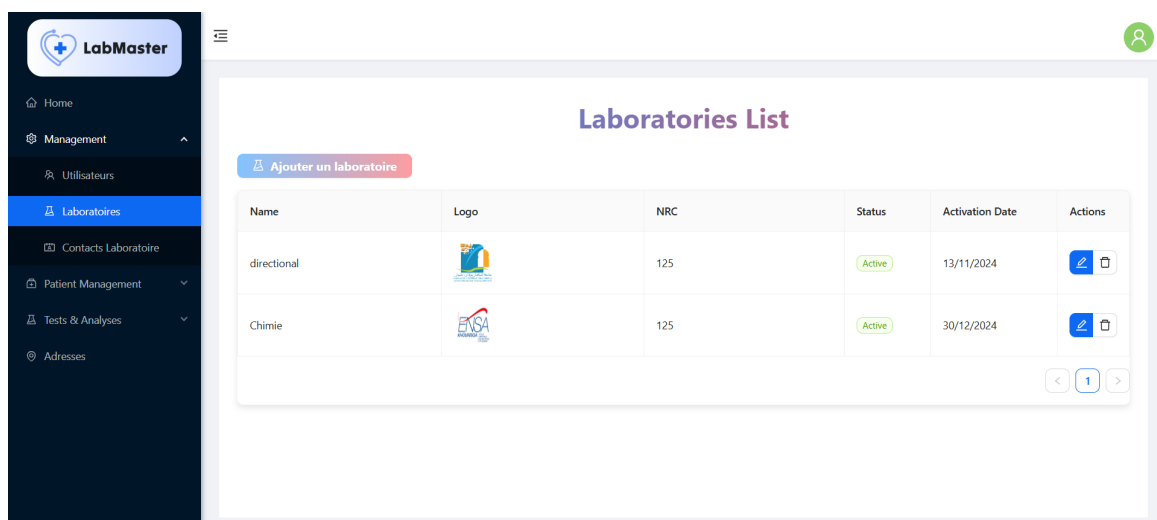


FIGURE 5.7 – Page de Gestion des Laboratoires

5.6 Contacts des Laboratoires

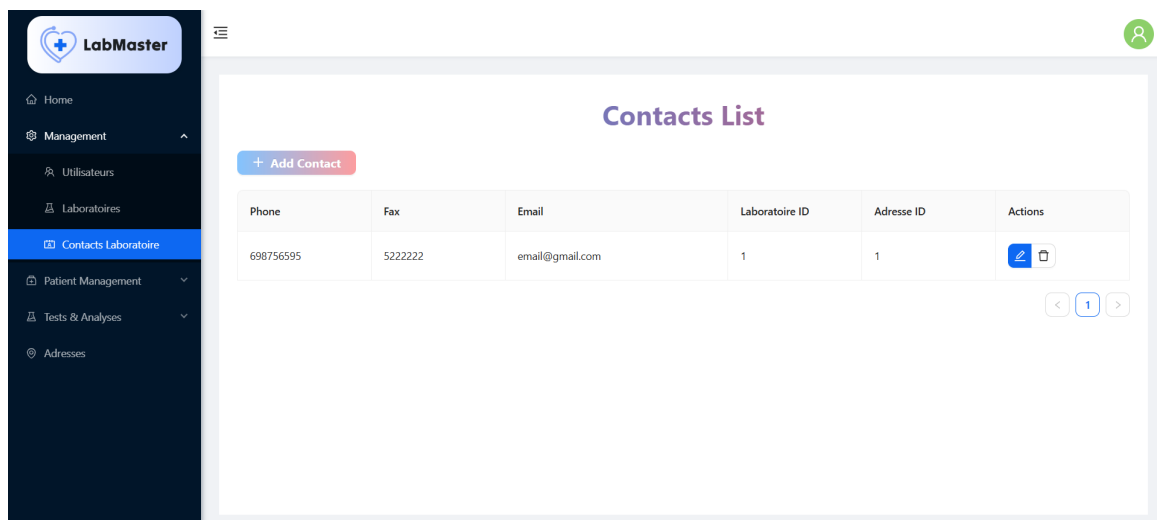


FIGURE 5.8 – Page des Contacts des Laboratoires

5.7 Gestion des Patients

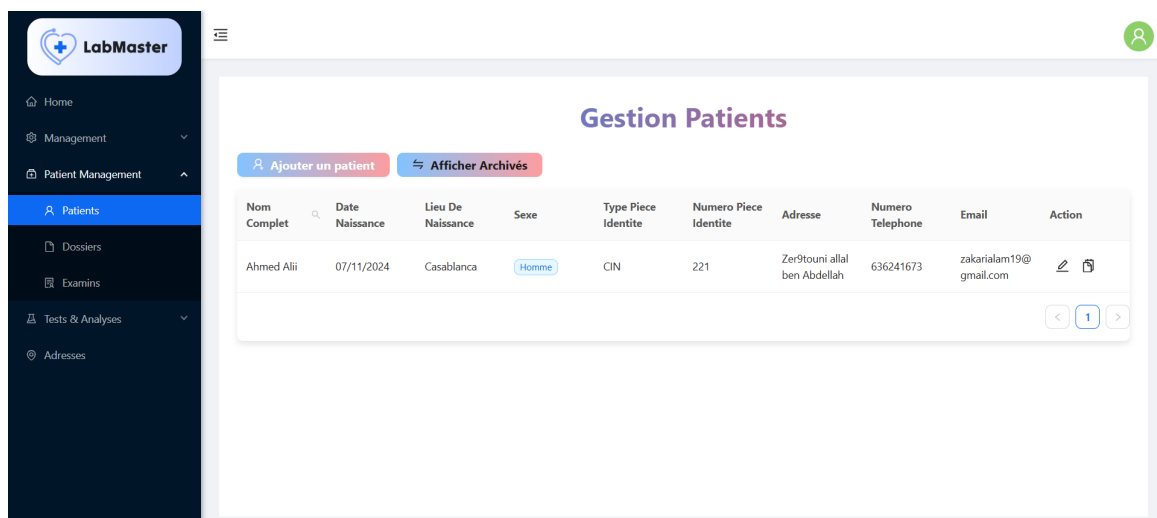


FIGURE 5.9 – Page de Gestion des Patients

5.8 Gestion des Dossiers

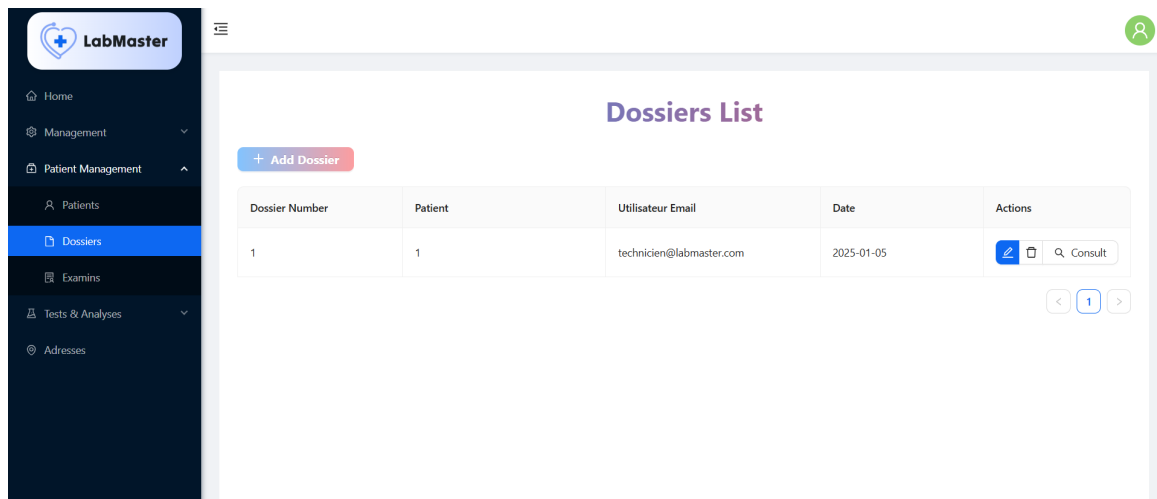


FIGURE 5.10 – Page de Gestion des Dossiers

5.9 Gestion des Examens

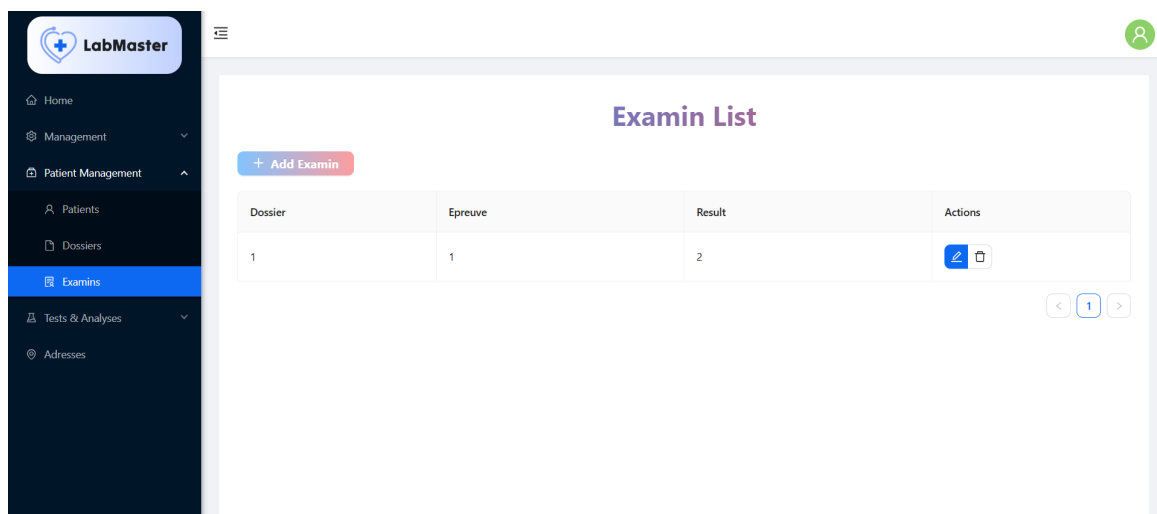


FIGURE 5.11 – Page de Gestion des Examens

5.10 Gestion des Épreuves

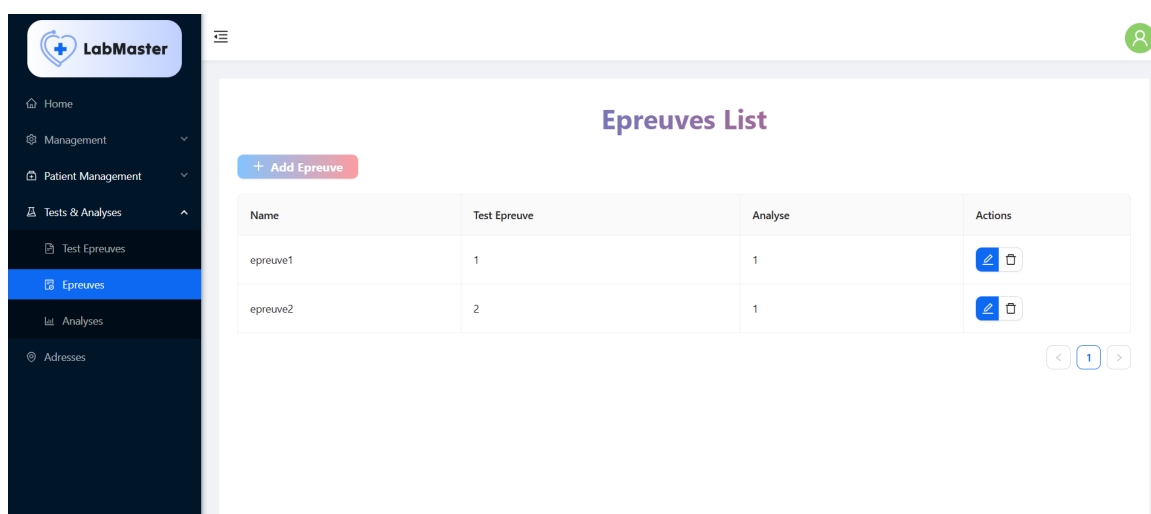


FIGURE 5.12 – Page de Gestion des Épreuves

5.11 Gestion des Tests D'épreuves

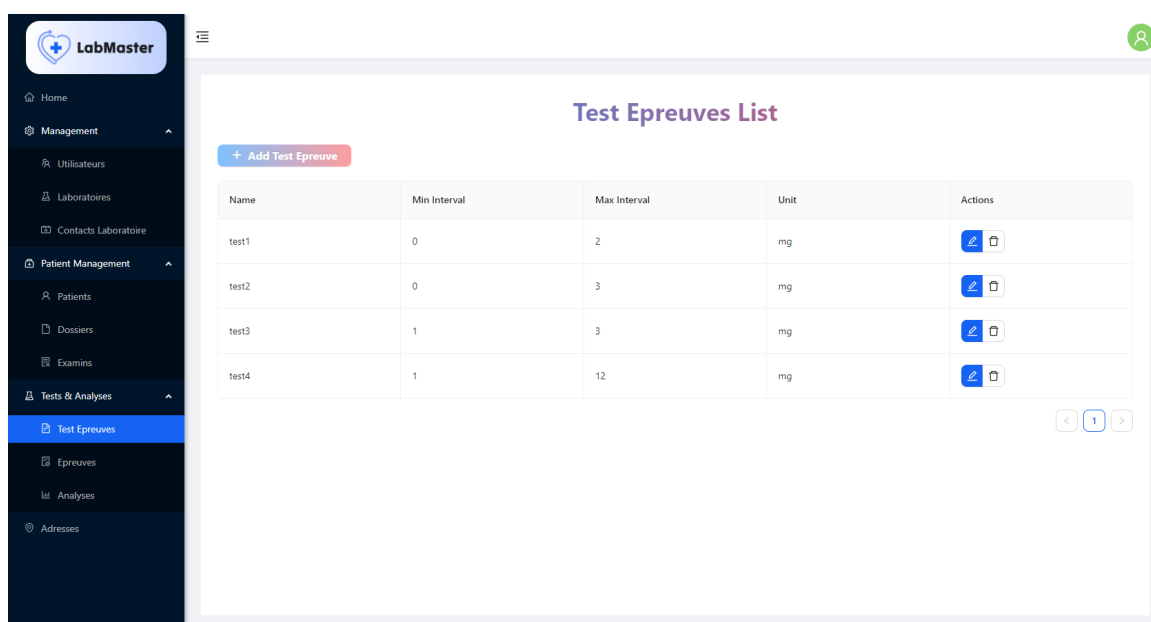


FIGURE 5.13 – Page de Gestion des Tests D'épreuves

5.12 Gestion des Adresses

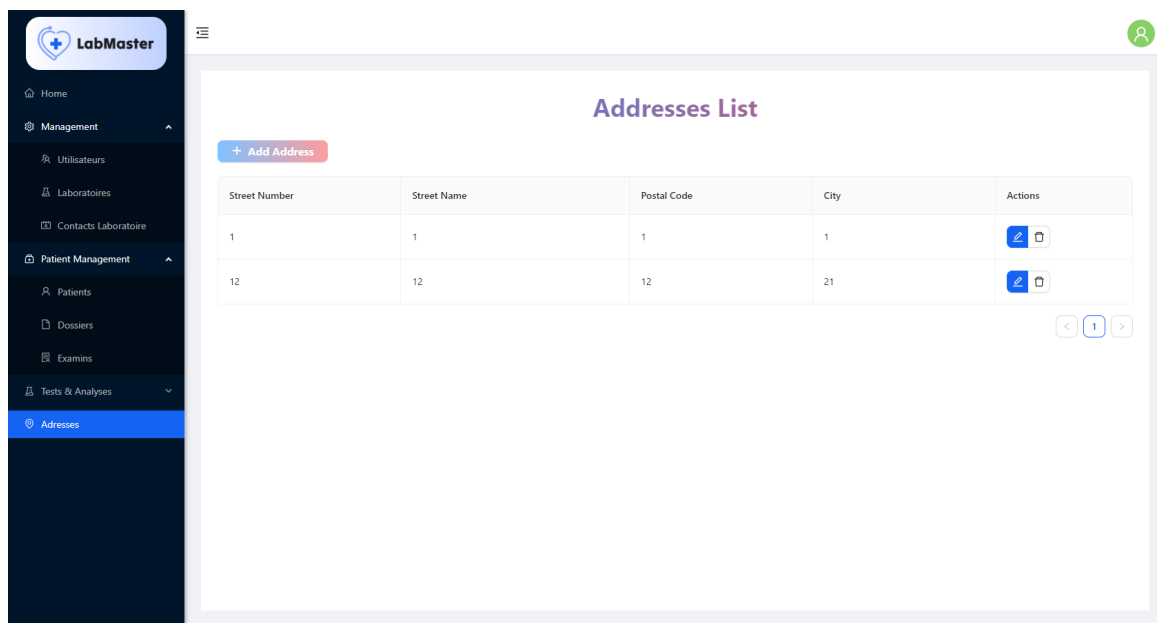


FIGURE 5.14 – Page de Gestion des Adresses

5.13 Consultation des Dossiers

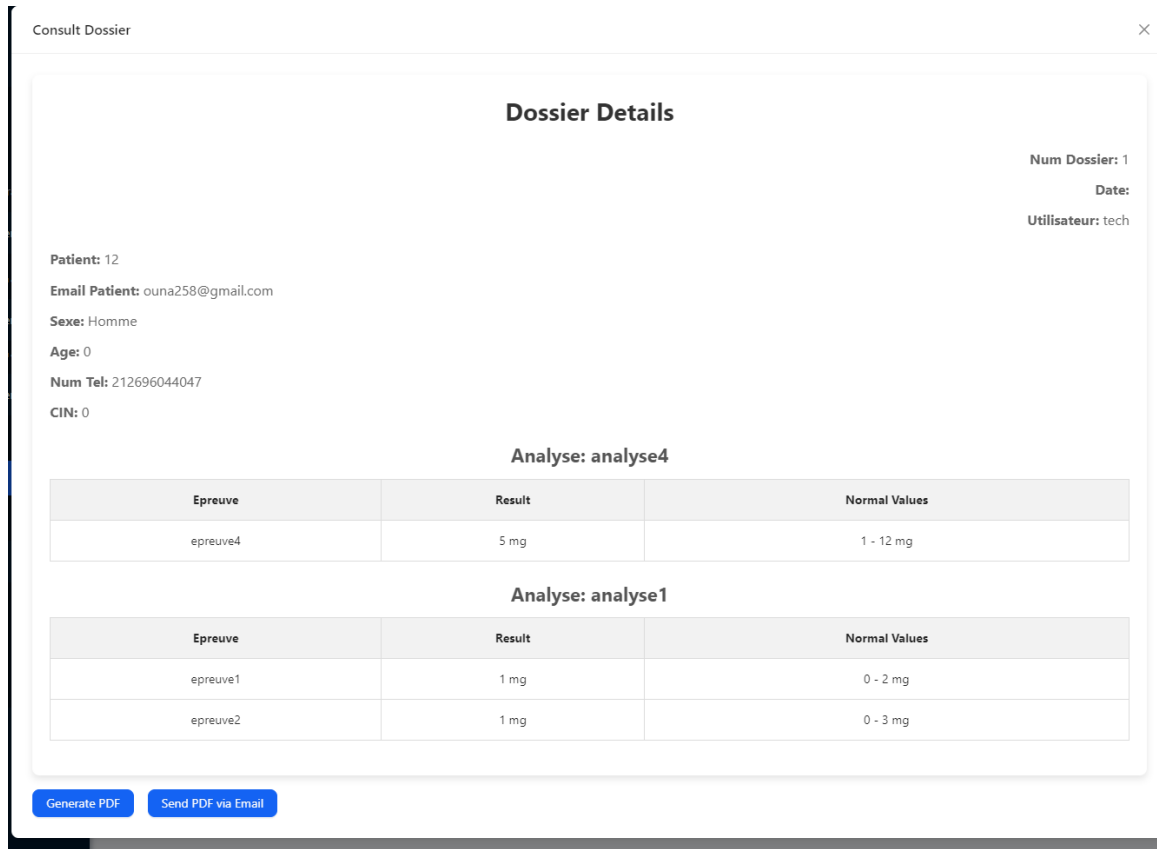
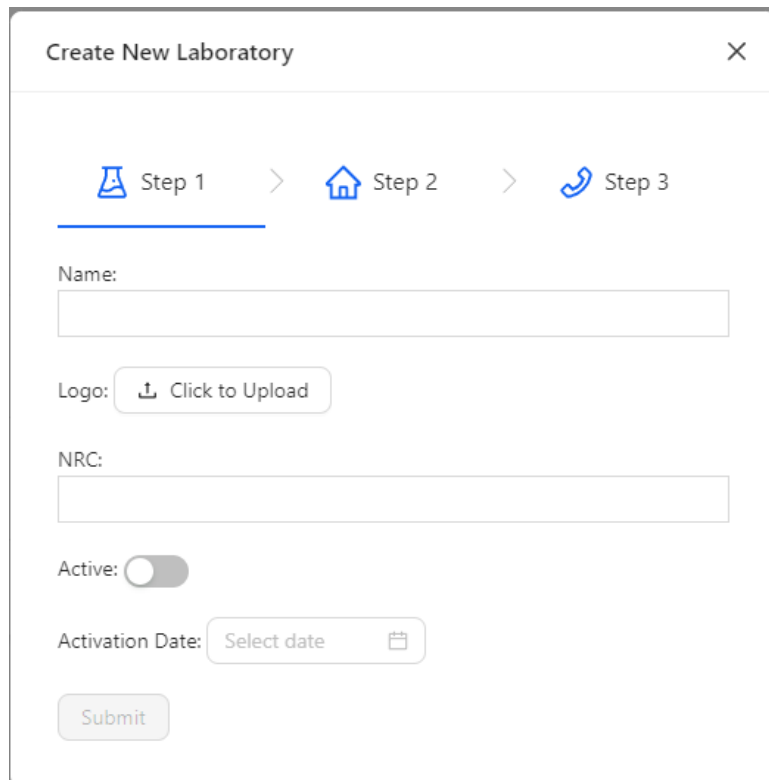


FIGURE 5.15 – Page de Consultation des Dossiers

5.14 Étapes dans les Laboratoires



The image shows a 'Create New Laboratory' form with a three-step progress indicator at the top. Step 1 (flask icon) is active, Step 2 (house icon) is next, and Step 3 (phone icon) is last. The form fields include: a 'Name' text input; a 'Logo' section with an upload button labeled 'Click to Upload'; an 'NRC' text input; an 'Active' toggle switch currently turned off; an 'Activation Date' section with a date picker button labeled 'Select date'; and a 'Submit' button at the bottom.

Create New Laboratory

Step 1 > Step 2 > Step 3

Name:

Logo: [Click to Upload](#)

NRC:

Active: ☐

Activation Date: [Select date](#)

Submit

FIGURE 5.16 – Étapes dans les Laboratoires

Chapitre 6

Conclusion et Perspectives

Au terme de ce projet, nous avons acquis des compétences significatives dans divers domaines essentiels au développement logiciel moderne. L'architecture microservices, mise en œuvre avec Spring Boot, ainsi que l'intégration de services tels que Discovery, Config et Gateway, nous ont permis de concevoir un système modulaire et scalable. La sécurité applicative avancée, incluant JWT, la gestion des rôles et la configuration CORS, a renforcé notre capacité à protéger les données sensibles et à gérer les accès de manière fine et sécurisée.

La conception d'interfaces dynamiques avec Angular et ng-zorro nous a doté des compétences nécessaires pour développer des applications front-end réactives et intuitives, répondant aux attentes des utilisateurs en termes d'expérience et de fonctionnalité. De plus, la maîtrise des tests unitaires et d'intégration, ainsi que la mise en place d'une pipeline CI/CD fiable avec GitHub Actions, Docker et SonarQube, ont assuré la qualité et la fiabilité continue de notre application.

Pour la suite, nous envisageons plusieurs axes d'amélioration et d'extension de notre projet :

- **Ajout de Fonctions d'Amélioration de l'Utilisabilité** : Intégrer des fonctionnalités telles que des cases à cocher sélectives, une recherche plus performante et des filtres avancés pour faciliter la navigation et la gestion des données par les utilisateurs.
- **Développement de Fonctionnalités Statistiques Avancées** : Ajouter des modules pour générer des rapports et des analyses statistiques détaillées, permettant une meilleure prise de décision basée sur les données collectées.
- **Déploiement sur un Cluster Kubernetes** : Migrer l'application vers un cluster Kubernetes afin d'assurer une haute disponibilité, une scalabilité automatique et une gestion simplifiée des déploiements en production.
- **Renforcement de l'Automatisation des Déploiements** : Adopter des approches d'infrastructure-as-code avec des outils tels que Terraform ou Ansible pour automatiser davantage les processus de déploiement, réduisant ainsi les risques d'erreurs humaines et accélérant le cycle de mise en production.

Ces perspectives nous offrent une feuille de route claire pour continuer à améliorer et à étendre notre application, en intégrant des technologies de pointe et en répondant aux besoins évolutifs des utilisateurs. En renforçant nos compétences techniques et en adoptant des pratiques de développement modernes, nous sommes bien préparés pour relever les défis futurs et contribuer de manière significative au domaine de la gestion des laboratoires et au-delà.

Bibliographie

- [1] Spring boot documentation. <https://spring.io/projects/spring-boot>, 2024.
- [2] Spring security documentation. <https://spring.io/projects/spring-security>, 2024.
- [3] Json web tokens (jwt). <https://jwt.io/>, 2024.
- [4] Angular documentation. <https://angular.io/docs>, 2024.
- [5] ng-zorro documentation. <https://ng.ant.design/docs/introduce/en>, 2024.
- [6] Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2024.
- [7] Sonarqube documentation. <https://www.sonarqube.org/documentation/>, 2024.
- [8] Apache jmeter user manual. <https://jmeter.apache.org/usermanual/index.html>, 2024.
- [9] Selenium documentation. <https://www.selenium.dev/documentation/>, 2024.
- [10] Docker documentation. <https://docs.docker.com/>, 2024.
- [11] Github actions documentation. <https://docs.github.com/en/actions>, 2024.
- [12] Mysql documentation. <https://dev.mysql.com/doc/>, 2024.
- [13] Swagger documentation. <https://swagger.io/docs/>, 2024.
- [14] Docker compose documentation. <https://docs.docker.com/compose/>, 2024.